

Grupo:

Danniel Angelo

Gustavo Silva Faustino

Kevin da Silva Lima

Arthur Vinicius Santos

Kauã Oliveira dos Santos

Lucas Calil Braga Gomes

Valmir Martins Dos Santos Junior

## **RPG DE TEXTO - BlooDivine:**

Brasil

2026



Grupo:  
Danniel Angelo  
Gustavo Silva Faustino  
Kevin da Silva Lima  
Arthur Vinicius Santos  
Kauã Oliveira dos Santos  
Lucas Calil Braga Gomes  
Valmir Martins Dos Santos Junior

## **RPG DE TEXTO - BlooDivine:**

Projeto Integrador 1

Centro Universitário Senac  
Análise e Desenvolvimento de Sistemas

Orientador: Marcus Gália

Brasil  
2026

# Resumo

O código apresentado desenvolve um pequeno RPG textual em Java utilizando conceitos fundamentais da programação orientada a objetos e interação via console. A aplicação simula um combate em turnos onde o jogador escolhe uma classe específica para enfrentar inimigos gerados pelo sistema. O projeto utiliza classes organizadas para separar responsabilidades, tornando o código mais modular e compreensível.

O jogador pode escolher entre Guerreiro, Assassino e Tank, cada um com atributos diferentes de vida, dano, cura e quantidade de poções, criando estilos distintos de jogabilidade. Após a escolha, o jogo inicia uma batalha contra inimigos como Esqueleto ou Goblin. Durante o combate, o jogador pode atacar, se curar ou visualizar seus status atuais.

O sistema também implementa recursos visuais simples para melhorar a experiência do usuário, como cores ANSI no terminal, efeito de digitação com atraso entre caracteres e limpeza de tela simulada. Além disso, há validação de entrada para impedir opções inválidas, aumentando a estabilidade da aplicação.

O combate ocorre em rodadas até que a vida do jogador ou do inimigo chegue a zero, demonstrando o uso de estruturas condicionais, laços de repetição, métodos e lógica básica de jogos em Java.

**Palavras-chave:** RPG. Java. Jogo textual. Algoritmos. Banco de dados.

## Lista de ilustrações



## Lista de quadros



## Lista de tabelas



# Sumário

|     |                                            |    |
|-----|--------------------------------------------|----|
| 1   | INTRODUÇÃO . . . . .                       | 11 |
| 1.1 | Objetivo . . . . .                         | 13 |
| 1.2 | Finalidade . . . . .                       | 14 |
| 2   | FLUXOGRAMA . . . . .                       | 15 |
| 2.1 | Versão Inicial . . . . .                   | 15 |
| 2.2 | Versão Final . . . . .                     | 20 |
| 2.3 | Banco de dados . . . . .                   | 25 |
| 3   | PORTUGUÊS ESTRUTURADO - PORTUGOL . . . . . | 33 |
| 3.1 | Versão Inicial . . . . .                   | 33 |
| 3.2 | Exemplo de Código em Portugol . . . . .    | 33 |
| 3.3 | Versão Final . . . . .                     | 37 |
| 4   | CÓDIGO FONTE . . . . .                     | 53 |
| 4.1 | Versão Inicial . . . . .                   | 53 |
| 4.2 | Versão Final . . . . .                     | 70 |
| 5   | CONCLUSÃO . . . . .                        | 91 |



# 1 Introdução

- Matérias

**Algoritmos e Programação** A disciplina de Algoritmos e Programação constitui um dos principais fundamentos para o desenvolvimento do nosso jogo de RPG utilizando a linguagem Java. Essa área do conhecimento nos fornece as bases da lógica computacional, permitindo que possamos organizar e estruturar as instruções que determinam o funcionamento do sistema.

No contexto do nosso projeto, os algoritmos são utilizados para definir a lógica das mecânicas do jogo, como o sistema de combate, a movimentação entre as salas, o cálculo de dano, o gerenciamento de atributos e as interações com o ambiente virtual. Por meio de estruturas de controle, como condicionais e laços de repetição, conseguimos programar o comportamento do jogo de forma dinâmica e organizada.

Além disso, a linguagem Java nos permite estruturar o código de forma organizada e eficiente, possibilitando a criação de elementos que representam os componentes do jogo, como personagens, inimigos e itens. Essa organização contribui para uma melhor compreensão do sistema, facilitando sua manutenção, atualização e expansão ao longo do desenvolvimento.

Dessa forma, a disciplina de Algoritmos e Programação desempenha um papel essencial na construção da lógica central do nosso jogo, permitindo que possamos transformar ideias e conceitos em funcionalidades concretas dentro do sistema.

**Banco de Dados** A disciplina de Banco de Dados é essencial na criação da tabela de fluxograma do projeto, pois auxilia o administrador a organizar de forma estruturada como as informações do jogo são armazenadas e circulam no sistema. Em um RPG, onde há grande volume de dados — como personagens, inventário e progresso do jogador —, é necessário definir claramente como esses dados se relacionam e em que momento são utilizados.

Nesse contexto, os conceitos de Banco de Dados ajudam a construir uma tabela de fluxograma mais clara e eficiente, representando corretamente o caminho das informações, desde o armazenamento até sua consulta e atualização. Isso facilita a visualização do administrador sobre os processos do sistema e evita inconsistências no tratamento dos dados.

Dessa forma, a disciplina contribui diretamente para a elaboração de um fluxograma organizado e funcional, garantindo que as informações sejam bem estruturadas e que o funcionamento do jogo ocorra de maneira lógica e confiável.

Interface no Terminal e Experiência do Usuário No desenvolvimento do nosso jogo de RPG, a interação com o usuário será realizada por meio do terminal, utilizando apenas a execução do código em Java. Dessa forma, não haverá uma interface gráfica tradicional, sendo necessário organizar todas as informações e interações diretamente por meio de textos exibidos na tela.

Nesse contexto, torna-se fundamental planejar cuidadosamente a forma como as informações serão apresentadas ao jogador, garantindo que os comandos, mensagens e opções estejam organizados de maneira clara e compreensível. A estruturação adequada dos textos no terminal permite que o usuário entenda facilmente as ações disponíveis, como movimentação e progressão no jogo.

Além disso, pretendemos utilizar recursos de coloração de texto no terminal para melhorar a visualização das informações. Essa estratégia possibilita destacar elementos importantes do jogo, como mensagens de combate, danos causados, avisos do sistema e resultados das ações do jogador. Por exemplo, cores diferentes podem ser utilizadas para representar ataques, mensagens de erro ou eventos importantes durante a partida.

Portanto, mesmo utilizando apenas o terminal como meio de interação, buscamos aplicar princípios de organização e clareza na apresentação das informações, utilizando a formatação e a coloração dos textos para proporcionar uma experiência de uso mais intuitiva e agradável para o jogador.

- Usuário mínimo e usuário perfeito

#### 1. Público-alvo

O público-alvo do nosso projeto são pessoas que gostam de jogos do gênero RPG, principalmente aqueles que se interessam por histórias de fantasia, evolução de personagens e tomadas de decisão durante o jogo. Como o nosso jogo será executado por meio do terminal, ele é voltado para usuários que possuem pelo menos uma familiaridade básica com o uso de computadores.

Além disso, buscamos atingir jogadores que gostam de experiências baseadas em leitura e escolhas, já que grande parte da interação do jogo acontece por meio de textos exibidos no terminal. Nosso objetivo é proporcionar uma experiência simples, mas envolvente, onde o jogador possa acompanhar a evolução do personagem e participar da história do jogo.

#### 2. Usuário mínimo

O usuário mínimo representa o perfil básico necessário para conseguir jogar o nosso jogo sem grandes dificuldades. Para isso, consideramos que o usuário precisa ter conhecimentos básicos sobre o uso do computador, como abrir programas, utilizar o teclado e interagir com o terminal.

Também é importante que o jogador seja alfabetizado, já que todas as ações do jogo dependem da leitura das mensagens que aparecem na tela e da escolha de opções durante a partida. Pensando nisso, estimamos uma idade mínima de aproximadamente 10 anos, pois nessa fase a maioria das pessoas já possui uma capacidade de leitura e compreensão suficiente para acompanhar o jogo.

Outro ponto importante é que o usuário mínimo precisa ter paciência, já que o jogo exige atenção para ler os textos e entender as situações apresentadas antes de tomar uma decisão.

### 3. Usuário perfeito

Para nós, o usuário perfeito do jogo seria um adolescente entre 14 e 17 anos que tenha interesse pelo gênero fantasia e goste de jogos de RPG. Esse tipo de jogador normalmente se envolve mais com a história, com os personagens e com os desafios que aparecem ao longo do jogo.

Além disso, acreditamos que o usuário ideal seja uma pessoa curiosa e paciente, que goste de explorar as possibilidades do jogo e acompanhar o desenvolvimento da narrativa com atenção.

Outro fator que contribui bastante é ter alguma experiência com RPG, seja em jogos digitais ou até mesmo em outros tipos de RPG. Isso ajuda o jogador a entender mais facilmente mecânicas como evolução de personagem, combate, itens e tomada de decisões estratégicas, permitindo que ele aproveite melhor a experiência que estamos propondo com o projeto.

## 1.1 Objetivo

O objetivo do nosso projeto é desenvolver um jogo de RPG textual em Java que funcione inteiramente no terminal, aplicando na prática os conceitos de lógica de programação e estrutura de sistemas. O jogo deverá permitir que o usuário crie seu personagem, tome decisões ao longo da narrativa e interaja com diferentes elementos do ambiente, como inimigos, salas e itens.

Além disso, buscamos implementar funcionalidades essenciais, como sistema de combate, movimentação entre cenários, gerenciamento de atributos e progressão do jogador. O projeto também tem como objetivo organizar essas funcionalidades de forma clara e eficiente, garantindo que o sistema funcione corretamente e seja de fácil entendimento.

Outro objetivo importante é estruturar a interação com o usuário de maneira intuitiva, utilizando textos bem organizados e recursos visuais no terminal, como cores e destaques, para melhorar a experiência durante o jogo.

## 1.2 Finalidade

A finalidade do nosso projeto é colocar em prática os conhecimentos adquiridos nas disciplinas de Algoritmos e Programação e Banco de Dados, demonstrando como esses conceitos podem ser utilizados no desenvolvimento de um sistema real.

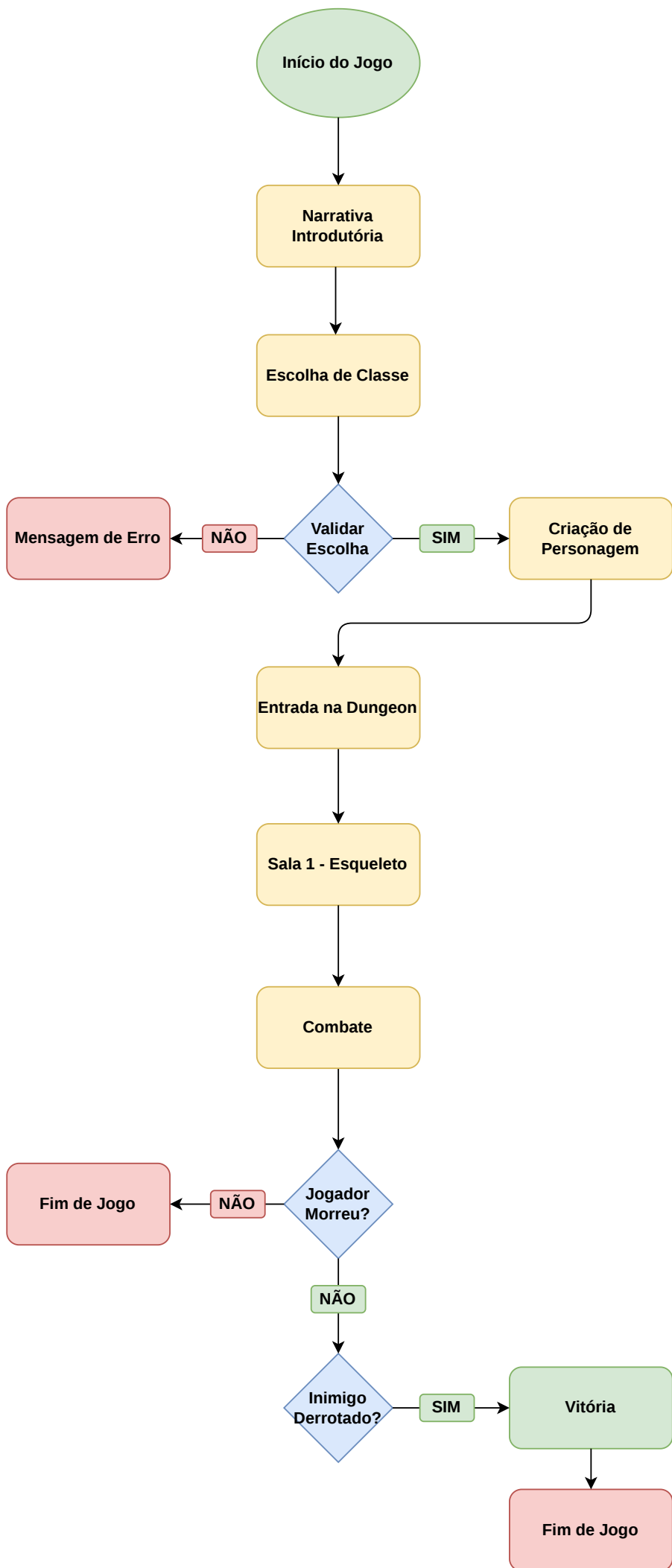
O projeto também tem como propósito oferecer uma experiência interativa ao usuário, onde ele possa acompanhar uma história, tomar decisões e evoluir seu personagem ao longo do jogo, mesmo em um ambiente simples como o terminal.

Além disso, buscamos desenvolver habilidades importantes para nossa formação, como raciocínio lógico, organização de código, resolução de problemas e trabalho em equipe.

Por fim, o jogo tem como finalidade servir como base para futuras melhorias, permitindo a adição de novas funcionalidades, expansão da narrativa e evolução das mecânicas, tornando o sistema cada vez mais completo e robusto.

## 2 Fluxograma

### 2.1 Versão Inicial



# Fluxograma

---

## Versão Inicial

---

Explicação do Diagrama (Fluxo da Versão Inicial)

### 1. Início do Jogo

Essa é a entrada do sistema. É o ponto onde o jogador inicia sua jornada no mundo de BLOODVINE.

### 1. Narrativa Introdutória

Nesta etapa, o sistema apresenta o contexto da história. O texto é exibido de forma imersiva no terminal, utilizando cores para destacar elementos importantes e preparar o jogador para a aventura.

### 1. Escolha de Classe

O jogador deve selecionar a classe do seu personagem entre as opções disponíveis:

- Guerreiro
- Assassino
- Tank

Essa fase define os atributos iniciais (vida, dano e cura) que serão utilizados durante o combate.

### 1. Validação de Escolha

O sistema verifica se a opção digitada pelo jogador é válida (números de 1 a 3).

- Se a escolha for inválida: O sistema exibe uma mensagem de erro e redireciona o jogador de volta para a tela de escolha de classe.
- Se a escolha for válida: O jogador avança para o combate.

### 1. Combate (Esqueleto)

Esta é a parte principal da versão inicial, onde ocorre o confronto contra o primeiro inimigo do jogo. O jogador visualiza o menu de ações com as seguintes opções:

#### Atacar

- O jogador causa dano ao inimigo.
- O inimigo realiza um contra-ataque imediato.

#### Curar

- O jogador utiliza uma poção para recuperar vida (se houver estoque).
- O inimigo aproveita o turno para atacar o jogador.

#### Status

- Exibe as estatísticas atuais do jogador, como vida restante e dano acumulado, sem avançar o turno de combate.

#### 1. Verificação de Morte e Resultado

Após cada ação de ataque ou cura, o sistema verifica se a vida de algum dos combatentes chegou a zero.

- Se o jogador morrer: O sistema exibe a mensagem “Você morreu” e o fluxo é encerrado.
- Se o inimigo morrer: O sistema exibe a mensagem “Você venceu” e o fluxo é encerrado.

#### 1. Fim do Jogo

Após a definição do resultado do combate:

- A sessão é finalizada.
- O fluxo do sistema é encerrado.

#### Resumo Geral

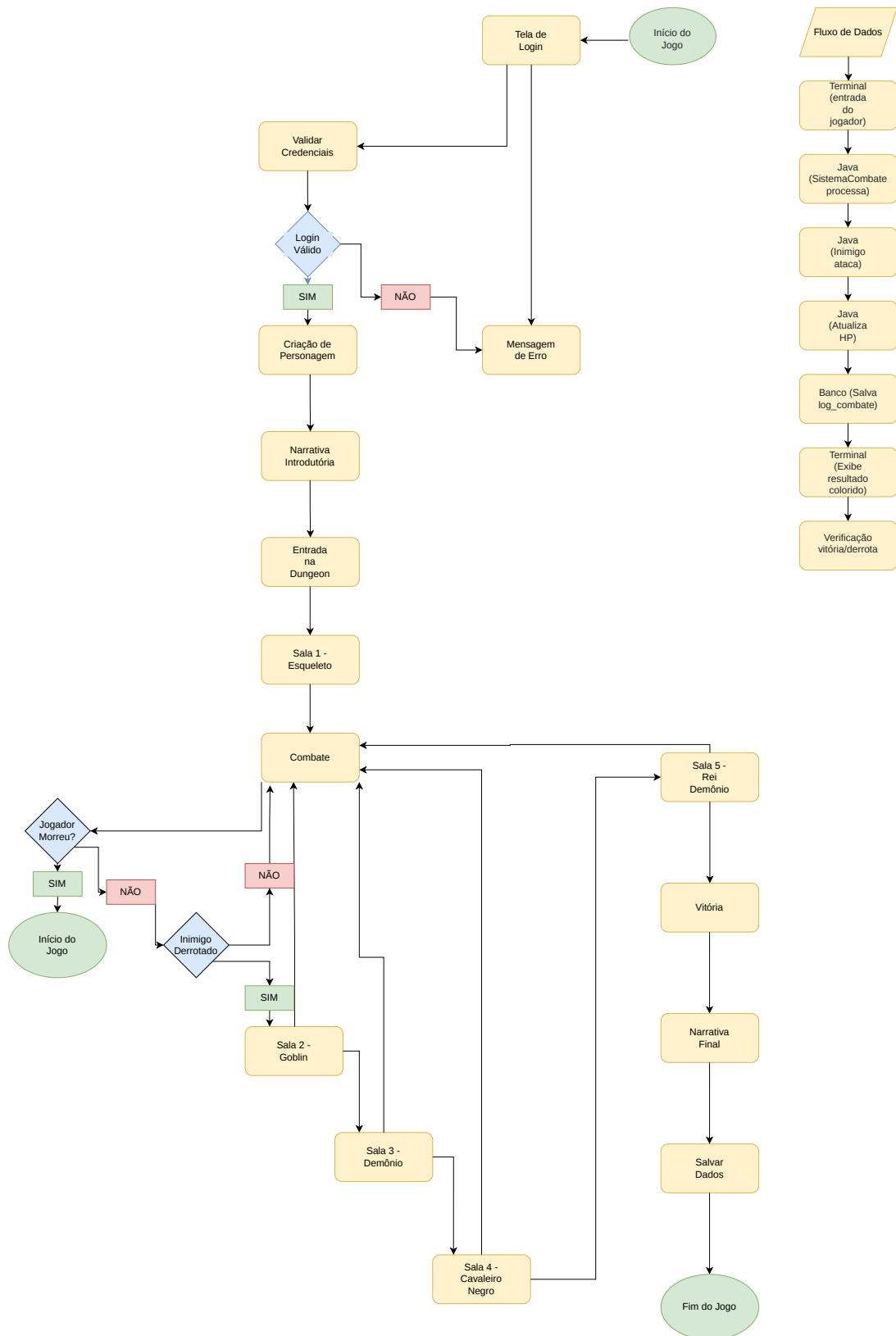
O diagrama representa o fluxo completo da versão inicial do RPG, desde o início da narrativa, passando pela escolha e validação da classe, o sistema de turnos do combate e finalizando com a verificação de vitória ou derrota do jogador.

#### Explicação do Diagrama (Fluxo de Dados)

O fluxo lateral representa como as informações são processadas tecnicamente pelo sistema:

- Terminal (entrada do jogador): O sistema recebe o comando via Scanner.
- Java (SistemaCombate processa): A lógica de jogo calcula as ações escolhidas.
- Java (Inimigo ataca): O objeto do inimigo executa sua lógica de dano.
- Java (Atualiza HP): Os valores de vida do Player e do Inimigo são atualizados na memória.
- Terminal (Exibe resultado colorido): O feedback visual é enviado ao usuário com cores ANSI.
- Verificação vitória/derrota: O sistema valida as condições de parada do loop de combate.

## 2.2 Versão Final



# Versão Final

---

## Explicação do Diagrama (Fluxo do Admin)

### 1. Início

Essa é a entrada do sistema. É o ponto onde o administrador começa a interação.

### 1. Tela de Login Admin

Nesta etapa, o administrador deve informar:

Usuário Senha

Essa fase tem como objetivo garantir a segurança e o controle de acesso ao sistema.

### 1. Validação de Login

O sistema verifica se as credenciais informadas estão corretas.

Se o login for inválido: O sistema exibe uma mensagem de erro O usuário é redirecionado de volta para a tela de login Se o login for válido: O usuário é direcionado para o Menu Admin

### 1. Menu Admin

Esta é a tela principal do administrador, onde estão disponíveis as funcionalidades de gerenciamento do sistema.

## Gerenciar Jogadores

Permite ao administrador:

- Criar jogadores
- Editar informações
- Remover jogadores

Essa funcionalidade garante o controle total sobre os usuários do sistema.

## Logs de Combate

Exibe o histórico das batalhas, incluindo:

- Participantes
- Resultados
- Informações relevantes

Essa área é importante para análise e monitoramento do sistema.

Gerenciar Inimigos

Permite:

- Criar inimigos
- Ajustar níveis de dificuldade
- Editar atributos

Essa função é essencial para o balanceamento do jogo.

Gerenciar Salas

Possibilita:

- Criar novas salas ou mapas
- Editar salas existentes
- Organizar o ambiente do jogo

Define onde as interações dos jogadores irão ocorrer.

Ver Estatísticas

Apresenta dados gerais do sistema, como:

- Desempenho dos jogadores
- Informações de combate
- Indicadores do sistema

Auxilia na análise e tomada de decisões.

Logout

Permite ao administrador encerrar sua sessão com segurança.

## 1. Fim

Após o logout:

- A sessão é finalizada
- O fluxo do sistema é encerrado

O diagrama representa o fluxo completo do administrador, desde o acesso ao sistema, passando pela validação de credenciais, utilização das funcionalidades administrativas e finalizando com o encerramento da sessão.

### Explicação do Diagrama (Fluxo Principal)

O nosso projeto consiste no desenvolvimento de um RPG de texto em Java, executado diretamente no terminal. A proposta do jogo é oferecer uma experiência baseada em narrativa e combate, utilizando apenas texto e cores para criar imersão no jogador. Nosso foco neste momento é definir o fluxo geral do jogo, ou seja, como o jogador irá percorrer cada etapa do sistema.

O jogo começa com um sistema de login, onde o jogador deverá informar um usuário e uma senha para acessar o sistema. Essa etapa inicial serve para identificar o jogador antes do início da aventura.

Após o login, o jogador será direcionado para a etapa de criação do personagem. Nesse momento, ele poderá definir algumas características básicas do seu personagem dentro do universo do jogo. Entre as opções disponíveis, o jogador poderá escolher uma classe de RPG, podendo optar entre Guerreiro, Tank ou Assassino. Além disso, também serão solicitadas informações como idade e gênero do personagem. Essas escolhas ajudam a personalizar a experiência do jogador dentro da história.

Depois da criação do personagem, iniciamos a introdução da história do jogo. Essa parte será apresentada em formato de texto narrativo, exibido lentamente no terminal para criar um efeito mais imersivo. Algumas partes do texto serão destacadas com cores, ajudando a enfatizar momentos importantes da narrativa e tornando a leitura mais dinâmica para o jogador.

Em seguida, o jogador será levado para a parte principal do jogo: as dungeons de combate. No MVP (Minimum Viable Product) do projeto, o sistema de combate será simples, oferecendo duas opções principais ao jogador durante as batalhas: atacar ou utilizar uma poção de cura. O objetivo é criar um sistema funcional de batalha que permita ao jogador enfrentar diferentes inimigos ao longo da progressão do jogo.

As dungeons serão organizadas em uma progressão de cinco salas, onde cada sala apresenta um inimigo diferente. À medida que o jogador avança, os inimigos se tornam mais fortes, aumentando gradualmente o nível de desafio. A ordem dos inimigos será a seguinte:

- Sala 1: Esqueleto
- Sala 2: Goblin
- Sala 3: Demônio
- Sala 4: Cavaleiro Negro
- Sala 5: Rei Demônio

Ao todo, o jogo será dividido em sete cenas principais, que representam o fluxo completo da experiência do jogador:

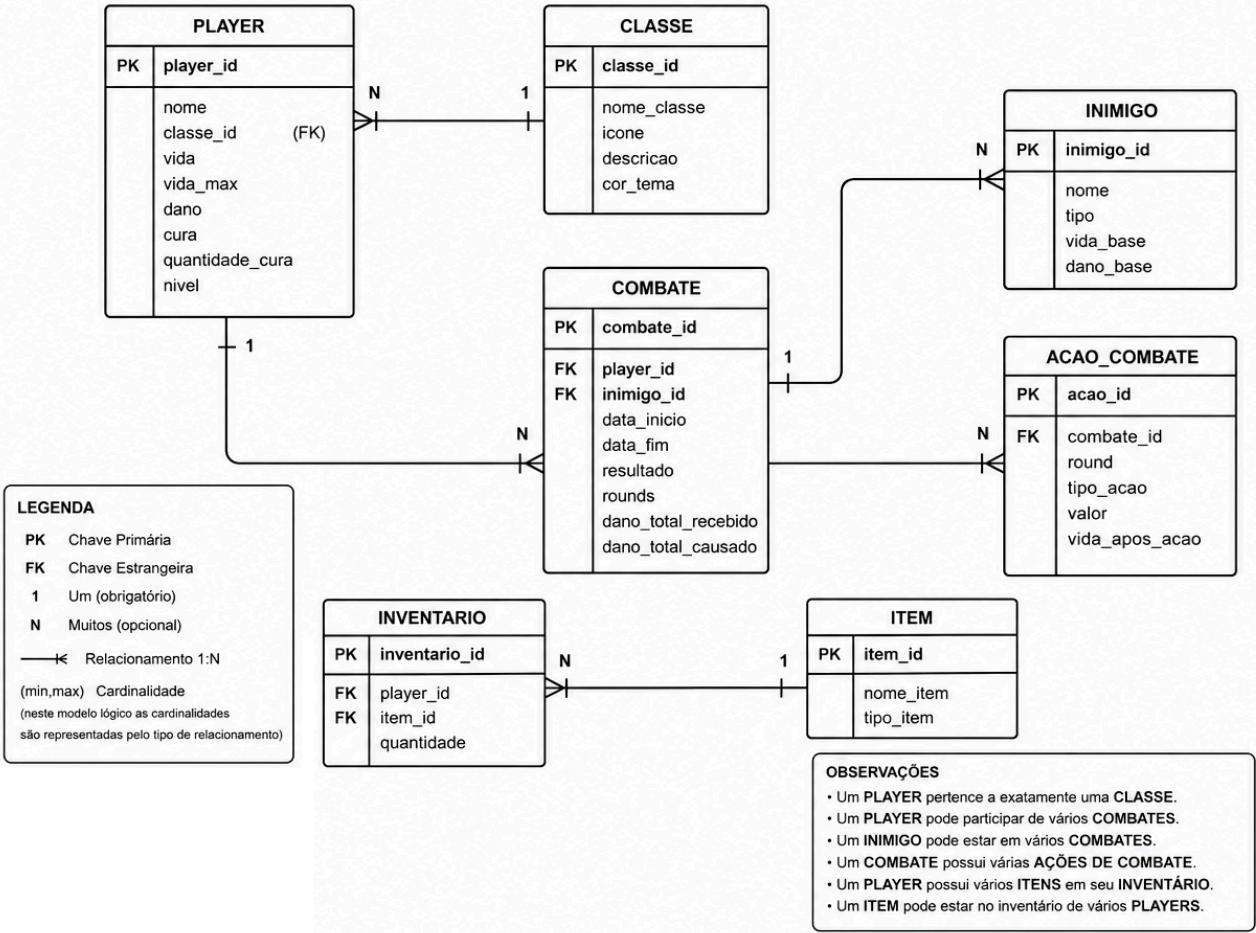
- Cena 1 – Introdução: apresentação da história e contextualização do jogador no mundo do jogo.
- Cena 2 – Combate contra o Esqueleto.
- Cena 3 – Combate contra o Goblin.
- Cena 4 – Combate contra o Demônio.
- Cena 5 – Combate contra o Cavaleiro Negro.
- Cena 6 – Combate contra o Rei Demônio.
- Cena 7 – Final: conclusão da história após o último combate.

Após derrotar o inimigo final, o jogador verá a parte final da narrativa, que servirá como encerramento da história do jogo. Assim como na introdução, essa parte também será apresentada em forma de texto narrativo, permitindo concluir a jornada do personagem dentro do universo que criamos.

Com essa estrutura, buscamos organizar de forma clara o fluxo do programa, mostrando como o jogador irá progredir desde o login inicial até o final da história, passando pela criação do personagem, introdução narrativa e sequência de combates nas dungeons. Nosso objetivo nesta etapa é apresentar a lógica geral da experiência do jogo, sem necessariamente entrar nos detalhes técnicos de implementação.

## 2.3 Banco de dados

BLOODVINE RPG - Modelo Lógico de Banco de Dados



# Banco de dados

---

## Modelo Lógico do Banco de Dados – BLOODVINE RPG

O modelo lógico do banco de dados do projeto BLOODVINE RPG foi desenvolvido para organizar e armazenar as principais informações do jogo, como jogadores, classes, inimigos, combates, ações realizadas durante as batalhas e inventário. A estrutura foi criada utilizando relacionamentos entre tabelas para garantir integridade e facilitar o gerenciamento dos dados do sistema.

A entidade central do modelo é a tabela PLAYER, responsável por armazenar os dados dos jogadores. Nela são registrados atributos importantes, como nome do personagem, vida atual, vida máxima, dano, quantidade de cura disponível e nível. Além disso, cada jogador possui uma chave estrangeira chamada classe\_id, que define a classe escolhida pelo jogador.

A tabela CLASSE armazena as classes disponíveis no RPG, como Guerreiro, Assassino e Tank. Cada classe possui informações próprias, como nome, ícone, descrição e cor temática. O relacionamento entre PLAYER e CLASSE é do tipo 1:N, ou seja, uma classe pode ser utilizada por vários jogadores, mas cada jogador pertence a apenas uma classe.

A tabela INIMIGO guarda os dados dos inimigos presentes no jogo. Entre os atributos armazenados estão nome, tipo, vida base e dano base. Os inimigos podem participar de vários combates ao longo do sistema.

A tabela COMBATE registra todas as batalhas realizadas no jogo. Ela funciona como uma entidade intermediária entre PLAYER e INIMIGO, armazenando informações como data de início e fim do combate, resultado da batalha, quantidade de rounds, dano total recebido e dano total causado. Cada combate está associado a um jogador e a um inimigo específico. Dessa forma, um jogador pode participar de vários combates, assim como um inimigo também pode aparecer em diferentes batalhas.

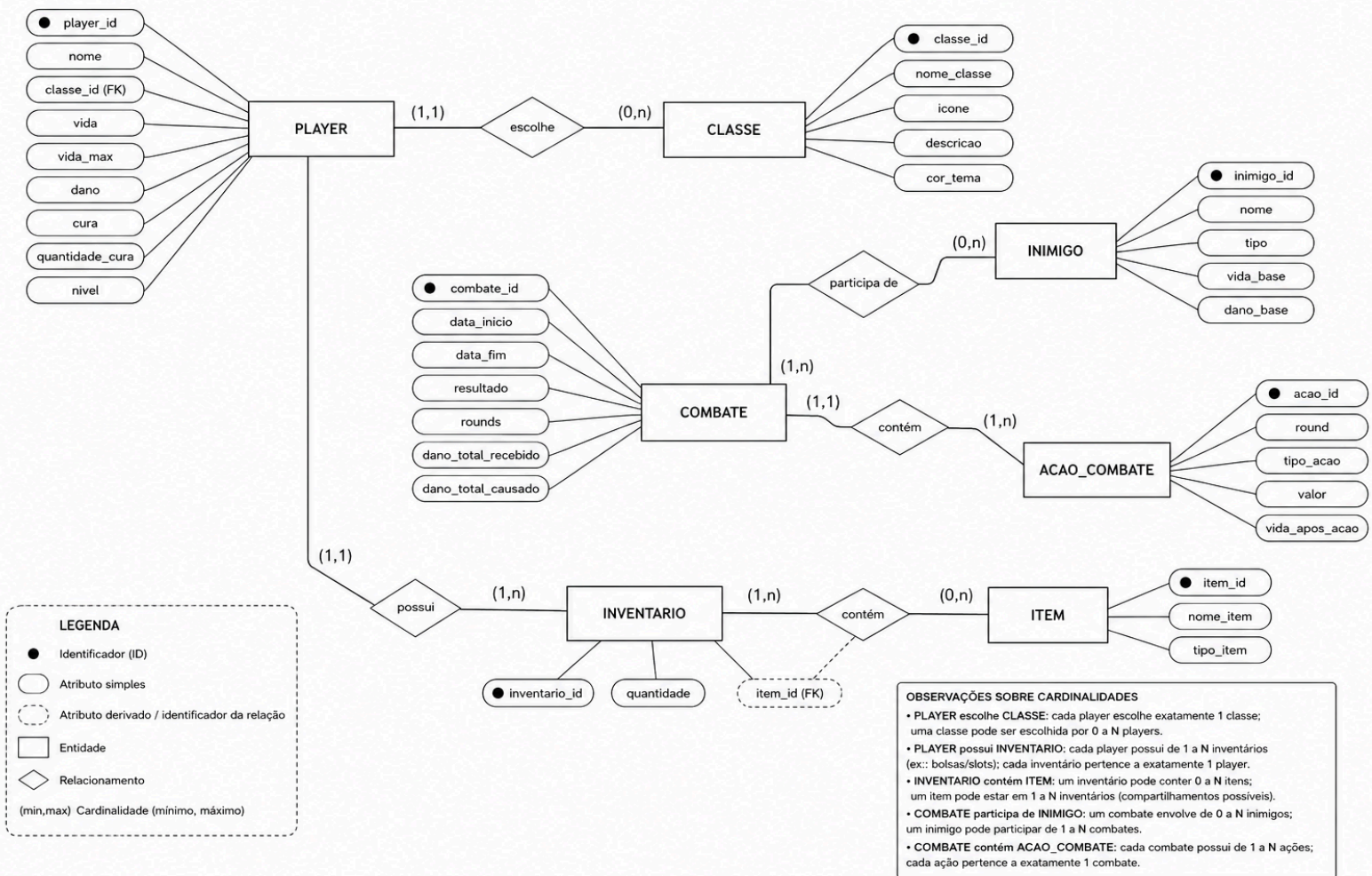
Durante as batalhas, as ações executadas são registradas na tabela ACAO\_COMBATE. Essa tabela armazena cada ação realizada em um combate, contendo informações como round da ação, tipo de ação executada, valor do dano ou cura e a vida restante após a ação. O relacionamento entre COMBATE e ACAO\_COMBATE é do tipo 1:N, permitindo que um combate possua várias ações registradas.

O sistema também possui um controle de inventário através da tabela INVENTARIO. Essa entidade faz a ligação entre os jogadores e os itens do jogo. Nela são armazenados o jogador dono do item, o item armazenado e sua quantidade. Isso permite que um jogador tenha vários itens e que um mesmo item possa existir no inventário de diferentes jogadores.

Por fim, a tabela ITEM armazena os dados dos itens disponíveis no RPG, como nome e tipo do item. O relacionamento entre ITEM e INVENTARIO é do tipo 1:N, pois um item pode aparecer em vários inventários.

Esse modelo lógico foi projetado para representar de forma organizada as mecânicas principais do BLOODVINE RPG, permitindo controlar personagens, batalhas e inventários de maneira eficiente, além de facilitar futuras expansões do sistema.

## BLOODVINE RPG - Modelo Conceitual de Banco de Dados



# Banco de dados

---

## Modelo Conceitual do Banco de Dados – BLOODVINE RPG

O modelo conceitual do banco de dados do projeto BLOODVINE RPG foi desenvolvido com o objetivo de representar de forma estruturada os principais elementos do jogo, permitindo o armazenamento e gerenciamento das informações relacionadas aos jogadores, classes, combates, inventário e itens. O modelo utiliza entidades, atributos e relacionamentos para organizar os dados de maneira lógica e eficiente.

A entidade **PLAYER** representa o jogador dentro do sistema. Nela são armazenadas informações como identificador do jogador (`player_id`), nome, vida atual, vida máxima, dano, capacidade de cura, quantidade de curas disponíveis e nível do personagem. Além disso, cada jogador possui uma chave estrangeira (`classe_id`) que o relaciona à entidade **CLASSE**.

A entidade **CLASSE** armazena os dados referentes às classes disponíveis no jogo, como Guerreiro, Assassino e Tank. Seus atributos incluem identificador da classe (`classe_id`), nome da classe, ícone representativo, descrição e cor temática. O relacionamento “escolhe” estabelece que cada jogador deve possuir exatamente uma classe, enquanto uma mesma classe pode ser escolhida por vários jogadores.

A entidade **COMBATE** representa as batalhas realizadas durante o jogo. Ela contém atributos como identificador do combate (`combate_id`), data de início, data de fim, resultado da batalha, quantidade de rounds, dano total recebido e dano total causado. Essa entidade permite registrar o histórico completo das lutas realizadas pelos jogadores.

Os inimigos enfrentados durante os combates são representados pela entidade **INIMIGO**, que possui atributos como identificador (`inimigo_id`), nome, tipo, vida base e dano base. O relacionamento “participa de” conecta os inimigos aos combates, permitindo que um combate envolva vários inimigos e que um mesmo inimigo possa aparecer em diferentes batalhas.

Durante os combates, diversas ações podem ocorrer, como ataques, curas e habilidades especiais. Essas ações são armazenadas na entidade **ACAO\_COMBATE**, composta pelos atributos identificador da ação (`acao_id`), round da ação, tipo da ação, valor causado e

vida após a ação. O relacionamento “contém” define que cada combate possui uma ou mais ações registradas, garantindo o acompanhamento detalhado de cada batalha.

O sistema também inclui a entidade INVENTARIO, responsável pelo armazenamento dos itens pertencentes aos jogadores. Cada inventário possui um identificador (inventario\_id) e a quantidade de itens armazenados. O relacionamento “possui” estabelece que cada jogador possui pelo menos um inventário, enquanto cada inventário pertence exclusivamente a um jogador.

Por fim, a entidade ITEM representa os itens disponíveis no jogo, contendo atributos como identificador (item\_id), nome do item e tipo do item. O relacionamento entre INVENTARIO e ITEM permite que um inventário contenha vários itens e que determinados itens possam existir em diferentes inventários.

Dessa forma, o modelo conceitual do BLOODVINE RPG organiza os dados essenciais do sistema de maneira estruturada, facilitando a implementação do banco de dados, a manutenção das informações e a expansão futura do projeto.



## 3 Português Estruturado - Portugal

### 3.1 Versão Inicial

[utf8]inputenc [T1]fontenc [brazil]babel  
color listings

### 3.2 Exemplo de Código em Portugal

Para mostrar o seu algoritmo sem erros de acentuação, use o ambiente `lstlisting`:

```

1
2 /*
3  Nome do Projeto : RPG Texto Bloodivine
4
5  Nome dos Integrantes do Grupo :
6  Nome : Gustavo Silva Faustino
7  Nome : Danniel Angelo Araujo Fonseca
8  Nome : Lucas Calil Braga Gomes
9  Nome : Kevin da Silva Lima
10 Nome : K a u Oliveira dos Santos
11 Nome : Valmir Martins Dos Santos Junior
12 Nome : Arthur Vinicius Santos
13
14 Data de criação do projeto : 12/02/2026
15 Data de Entrega do Projeto : 01/05/2026
16
17 */
18 programa {
19     inclua biblioteca Util --> u
20     inclua biblioteca Texto --> txt
21
22     funcao inicio() {
23         inteiro player_vida = 0
24         inteiro player_dano = 0
25         inteiro player_vidaMax = 0
26         cadeia player_classe = ""
27         inteiro quantidadeCura = 3
28         inteiro numRandom = 0
29
30         cadeia inimigo_nome = "ESQUELETO"
31         inteiro inimigo_vida = 50
32         inteiro inimigo_dano = 10

```

```
33
34     inteiro opcao
35
36
37     digitar("--- BEM VINDO AO RPG ---\n")
38     escreva("Escolha: 1-GUERREIRO | 2-ASSASSINO | 3-TANK\n> ")
39     leia(opcao)
40     enquanto(opcao < 1 ou opcao > 3){
41         digitar("Opção inválida!\n")
42         escreva("Escolha: 1-GUERREIRO | 2-ASSASSINO | 3-TANK\n> ")
43         leia(opcao)
44     }
45
46     se (opcao == 1) {
47         player_vida = 100
48         player_dano = 20
49         player_classe = "GUERREIRO"
50         player_vidaMax = 100
51     }
52     senao se(opcao == 3) {
53         player_vida = 120
54         player_dano = 15
55         player_classe = "TANK"
56         player_vidaMax = 120
57     }
58     senao se(opcao == 2){
59         player_vida = 80
60         player_dano = 21
61         player_classe = "ASSASSINO"
62         player_vidaMax = 80
63     }
64
65     escreva("\nUm ", inimigo_nome, " apareceu!\n")
66
67     u.aguarde(1000)
68
69     enquanto (player_vida > 0 e inimigo_vida > 0) {
70         escreva("\nSua Vida: ", player_vida, " | Vida do Inimigo: ",
71             inimigo_vida, "\n")
72         digitar("O que deseja fazer? 1-Atacar | 2-Curar | 3-Sair\n> ")
73         leia(opcao)
74
75         enquanto(opcao < 1 ou opcao > 3){
76             digitar("Opção inválida!\n")
77             escreva("\nSua Vida: ", player_vida, " | Vida do Inimigo: ",
78                 inimigo_vida, "\n")
79             digitar("O que deseja fazer? 1-Atacar | 2-Curar | 3-Sair\n> ")
```

```
78     leia(opcao)
79 }
80
81 se (opcao == 1) {
82     numRandom = u.sorteia(0, 2)
83     se (numRandom == 0) {
84         digitar("voc errou o ataque...\n")
85     }
86     senao se (numRandom == 1) {
87         digitar("voc acertou o ataque\n")
88     }
89
90     senao se (numRandom == 2) {
91         digitar("ATAQUE CRÍTICO!!!\n")
92     }
93     inimigo_vida -= player_dano * numRandom
94
95     se (inimigo_vida > 0) {
96         numRandom = u.sorteia(0, 2)
97         inteiro novoDanoInimigo = inimigo_dano * numRandom
98         se (numRandom == 0) {
99             digitar("o inimigo errou o ataque!\n")
100         }
101         senao se (numRandom == 1) {
102             digitar("o inimigo acertou o ataque...")
103         }
104         senao se (numRandom == 2) {
105             digitar("O INIMIGO ACERTOU UM ATAQUE CRÍTICO!!!\n")
106             escreva("O inimigo revidou e te causou ",
107                 novoDanoInimigo, " de dano!\n")
108         }
109         player_vida -= novoDanoInimigo
110     } senao {
111         escreva("Voc derrotou o ", inimigo_nome, "!\n")
112     }
113 }
114 senao se (opcao == 2) {
115
116     se (quantidadeCura > 0) {
117         player_vida += 30
118         quantidadeCura--
119         digitar("voc se curou em 30 pontos\n")
120     }
121     senao {
122         digitar("voc não tem mais cura...\n")
123     }
```

```
124
125
126     se(player_vida > player_vidaMax) {
127         player_vida = player_vidaMax
128     }
129     se (inimigo_vida > 0) {
130         player_vida -= inimigo_dano + (inimigo_dano * numRandom)
131         escreva("O inimigo te causou ", inimigo_dano, " de dano!\n")
132     }
133
134
135     } senao {
136         pare
137     }
138 }
139
140     digitar("\n--- FIM DE JOGO ---\n")
141 }
142 funcao digitar(cadeia texto) {
143     inteiro i = 0
144     inteiro tamanho = txt.numero_caracteres(texto)
145     enquanto(i < tamanho) {
146         escreva(txt.obter_caracter(texto, i))
147         u.aguarde(20)
148         i++
149     }
150
151 }
```

Listing 3.1 – Fim do Código em Portugol

## 3.3 Versão Final

```
1 programa {
2     inclua biblioteca Util --> u
3     inclua biblioteca Texto --> txt
4     inclua biblioteca Tipos --> t
5
6     // Legenda
7
8     // Globais
9     inteiro statusPlayer[4] = {0, 0, 0, 0}
10    inteiro statusInimigo[3] = {0, 0, 0}
11
12    cadeia nomeInimigos[6] = {"", "", "", "", "", ""}
13    cadeia nomePlayer[3] = {"", "", ""}
14
15    funcao inicio() {
16
17        inteiro opcao
18
19        digitar("==== BLOODVINE RPG ====\\n")
20        digitar("1 - Jogar\\n")
21        digitar("2 - Admin\\n")
22        digitar("3 - Sair\\n")
23
24        opcao = lerInteiro("> ")
25
26        enquanto(opcao < 1 ou opcao > 3){
27            digitar("Opção inválida!\\n")
28            opcao = lerInteiro("> ")
29        }
30
31        se(opcao == 1){
32            iniciarJogo()
33        }
34        senao se(opcao == 2){
35
36            se(loginAdmin()){
37                menuAdmin()
38            }
39            senao{
40                digitar("Login inválido!\\n")
41            }
42        }
43    }
44    senao{
45        digitar("Saindo do jogo...\\n")
46    }
```

```
47 }
48
49 // Iniciar Jogos
50
51 funcao iniciarJogoAdmin(inteiro classe, inteiro inimigo) {
52     criarPlayer(classe)
53     criarInimigo(inimigo)
54     combate(inimigo)
55     menuAdmin()
56 }
57
58 funcao iniciarJogo(){
59
60     // Fluxo de modo Boss Rush
61     escolherClasse()
62
63     // Esqueleto
64     criarInimigo(1)
65     mostrarSala(1)
66     combate(1)
67     resetClasse()
68
69     // Goblin
70     mostrarSala(2)
71     criarInimigo(2)
72     combate(2)
73     resetClasse()
74
75     // Dem nio
76     mostrarSala(3)
77     criarInimigo(3)
78     combate(3)
79     resetClasse()
80
81     // Cavaleiro Negro
82     mostrarSala(4)
83     criarInimigo(4)
84     combate(4)
85     resetClasse()
86
87     // Rei Dem nio
88     mostrarSala(5)
89     criarInimigo(5)
90     combate(5)
91
92     digitar("\nPARABÉNS! VOC   ZEROU O BLOODVINE RPG!\n")
93 }
```

```
94
95 // Login
96
97 funcao logico loginAdmin() {
98
99     cadeia nome = ""
100     cadeia senha = ""
101
102     escreva("Digite o seu nome:\n")
103     leia(nome)
104
105     escreva("Digite a sua senha:\n")
106     leia(senha)
107
108     se(nome == "admin" e senha == "admin123"){
109
110         digitar("Bem vindo ao modo admin...\n")
111         digitar("Aqui voc  pode modificar a classe do jogador e do
            inimigo\n")
112
113         retorne verdadeiro
114     }
115
116     retorne falso
117 }
118
119 funcao menuAdmin(){
120
121     inteiro opcao = 0
122     inteiro classe = 1
123     inteiro inimigo = 1
124
125     enquanto(opcao != 4){
126
127         limparTela()
128
129         digitar("==== MENU ADMIN =====\n")
130         digitar("1 - Escolher Classe\n")
131         digitar("2 - Escolher Inimigo\n")
132         digitar("3 - Iniciar Jogo\n")
133         digitar("4 - Sair\n")
134
135         opcao = lerInteiro("> ")
136
137         enquanto(opcao < 1 ou opcao > 4){
138             digitar("Opção inválida!\n")
139             opcao = lerInteiro("> ")
140         }
141     }
142 }
```

```
140     }
141
142     se(opcao == 1){
143
144         classe = escolherClasseAdmin()
145
146     }
147     senao se(opcao == 2){
148
149         inimigo = escolherInimigoAdmin()
150
151     }
152     senao se(opcao == 3){
153
154         digitar("Iniciando jogo...\n")
155         u.aguarde(1000)
156
157         iniciarJogoAdmin(classe, inimigo)
158
159     }
160     senao{
161
162         digitar("Saindo do menu admin...\n")
163         retorne
164     }
165 }
166 }
167
168
169
170 // Classes
171 funcao resetClasse() {
172     inteiro opcao
173     digitar("Gostaria de continuar com a mesma classe?\n")
174     digitar("Então escolha a mesma classe novamente...\n")
175
176     opcao = lerInteiro(" Escolha : 1 - GUERREIRO | 2 - ASSASSINO | 3 -
177         TANK \n > ")
178     limparTela()
179
180     enquanto (opcao < 1 ou opcao > 3) {
181         digitar(" Opção inválida! \n")
182         opcao = lerInteiro(" Escolha : 1 - GUERREIRO | 2 - ASSASSINO | 3
183             - TANK \n > ")
184         limparTela()
185     }
```

```
185     criarPlayer(opcao)
186     digitar("\nVOC     ESCOLHEU A CLASSE " + nomePlayer[opcao-1] +
187           "!!!\n")
187     limparTela()
188 }
189
190 funcao escolherClasse() {
191     digitar(" --- BEM VINDO AO RPG --- \n")
192     inteiro opcao
193     opcao = lerInteiro(" Escolha : 1 - GUERREIRO | 2 - ASSASSINO | 3 -
194           TANK \n > ")
194     limparTela()
195
196     enquanto (opcao < 1 ou opcao > 3) {
197         digitar(" Opção inválida! \n")
198         opcao = lerInteiro(" Escolha : 1 - GUERREIRO | 2 - ASSASSINO | 3
199           - TANK \n > ")
199         limparTela()
200     }
201
202     criarPlayer(opcao)
203     digitar("\nVOC     ESCOLHEU A CLASSE " + nomePlayer[opcao-1] +
204           "!!!\n")
204     u.aguarde(600)
205 }
206
207 funcao inteiro escolherClasseAdmin() {
208     digitar(" --- BEM VINDO A BLOODVINE ADMIN --- \n")
209     inteiro opcao
210     opcao = lerInteiro(" Escolha : 1 - GUERREIRO | 2 - ASSASSINO | 3 -
211           TANK | 4 - STATUS \n > ")
211     limparTela()
212
213     enquanto (opcao < 1 ou opcao > 4) {
214         digitar(" Opção inválida! \n")
215         opcao = lerInteiro(" Escolha : 1 - GUERREIRO | 2 - ASSASSINO | 3
216           - TANK | 4 - STATUS \n > ")
216         limparTela()
217     }
218     se (opcao == 4) {
219         mostrarStatusAdmin()
220         limparTelaRapido()
221     }
222     senao {
223         criarPlayer(opcao)
224         digitar("\nVOC     ESCOLHEU A CLASSE " + nomePlayer[opcao-1] +
225               "!!!\n")
```

```
225     u.aguarde(600)
226     retorne opcao
227 }
228 }
229
230
231 funcao inteiro escolherInimigoAdmin() {
232     digitar(" --- BEM VINDO A BLOODVINE ADMIN --- \n")
233     inteiro opcao
234     opcao = lerInteiro(" Escolha : 1 - ESQUELETO | 2 - GOBLIN | 3 -
        DEM NIO | 4 - CAVALEIRO NEGRO | 5 - REI DEM NIO | 6 STATUS \n
        > ")
235     limparTela()
236
237     enquanto (opcao < 1 ou opcao > 6) {
238         digitar(" Opção inválida! \n")
239         opcao = lerInteiro(" Escolha : 1 - ESQUELETO | 2 - GOBLIN | 3 -
            DEM NIO | 4 - CAVALEIRO NEGRO | 5 - REI DEM NIO | 6 -
            STATUS \n > ")
240         limparTela()
241     }
242     se(opcao == 6) {
243         mostrarStatusInimigosAdmin()
244     }
245     senao{
246         criarInimigo(opcao)
247         digitar("\nVOC  ESCOLHEU O INIMIGO " + nomeInimigos[opcao] +
            "!!!\n")
248         u.aguarde(600)
249         retorne opcao
250     }
251 }
252 }
253
254 funcao criarPlayer(inteiro classe) {
255     se (classe == 1) {
256         statusPlayer[0] = 100 // Vida
257         statusPlayer[1] = 20  // Dano
258         statusPlayer[2] = 5   // Velocidade
259         statusPlayer[3] = 5   // Cura
260         nomePlayer[0] = "GUERREIRO"
261     }
262     senao se (classe == 2) {
263         statusPlayer[0] = 50
264         statusPlayer[1] = 25
265         statusPlayer[2] = 6
266         statusPlayer[3] = 3
```

```
267     nomePlayer[1] = "ASSASSINO"
268 }
269 senao se (classe == 3) {
270     statusPlayer[0] = 120
271     statusPlayer[1] = 17
272     statusPlayer[2] = 2
273     statusPlayer[3] = 2
274     nomePlayer[2] = "TANK"
275 }
276 }
277
278 funcao criarInimigo(inteiro tipo) {
279     se (tipo == 1) { // Esqueleto
280         statusInimigo[0] = 50 // Vida
281         statusInimigo[1] = 15 // Dano
282         statusInimigo[2] = 2 // Velocidade
283         nomeInimigos[1] = "ESQUELETO"
284     }
285     senao se (tipo == 2) { // Goblin
286         statusInimigo[0] = 60
287         statusInimigo[1] = 15
288         statusInimigo[2] = 4
289         nomeInimigos[2] = "GOBLIN"
290     }
291     senao se (tipo == 3) { // Dem nio
292         statusInimigo[0] = 60
293         statusInimigo[1] = 20
294         statusInimigo[2] = 4
295         nomeInimigos[3] = "DEM NIO"
296     }
297     senao se (tipo == 4) { // Cavaleiro Negro
298         statusInimigo[0] = 100
299         statusInimigo[1] = 20
300         statusInimigo[2] = 5
301         nomeInimigos[4] = "(BOSS) CAVALEIRO NEGRO"
302     }
303     senao se (tipo == 5) { // Rei Dem nio
304         statusInimigo[0] = 130
305         statusInimigo[1] = 23
306         statusInimigo[2] = 5
307         nomeInimigos[5] = "(FINAL BOSS) REI DEM NIO"
308     }
309 }
310
311
312 funcao inteiro lerInteiro(cadeia mensagem) {
313     cadeia entradaTexto
```

```

314     inteiro numeroConvertido = 0
315     logico valido = falso
316
317     enquanto (valido == falso) {
318         digitar(mensagem)
319         leia(entradaTexto)
320         limparTelaRapido()
321
322
323         se (txt.numero_caracteres(entradaTexto) < 10 e
324             t.cadeia_e_inteiro(entradaTexto, 10)) {
325             numeroConvertido = t.cadeia_para_inteiro(entradaTexto, 10)
326             valido = verdadeiro
327         }
328         senao {
329             limparTelaRapido()
330             digitar("ERRO! Digite apenas números inteiros válidos (sem
331                 letras ou números gigantes, seu engraçadinho).\n\n")
332             u.aguarde(1000)
333             limparTelaRapido()
334         }
335     }
336     retorne numeroConvertido
337 }
338
339 // Fluxo de Combate
340 funcao combate(inteiro indiceInimigo) {
341     inteiro numRandom
342     inteiro vidaPlayerMax = statusPlayer[0]
343     inteiro opcao
344
345     escreva("\nUM " + nomeInimigos[indiceInimigo] + " APARECEU!!!\n")
346     digitar("COMEÇA O COMBATE!\n")
347     u.aguarde(1000)
348     limparTela()
349
350     enquanto (statusPlayer[0] > 0 e statusInimigo[0] > 0) {
351         escreva("\n-----")
352         escreva("\nSua Vida : ", statusPlayer[0], " | Vida do Inimigo :
353             ", statusInimigo[0], "\n")
354
355         opcao = lerInteiro("O que deseja fazer?\n 1 - Atacar | 2 - Curar
356             | 3 - Status \n > ")
357         limparTela()
358
359         enquanto (opcao < 1 ou opcao > 3) {
360             digitar(" Opção inválida! \n")

```

```
357     u.aguarde(1000)
358     escreva("\n-----")
359     escreva("\nSua Vida : ", statusPlayer[0], " | Vida do Inimigo :
        ", statusInimigo[0], "\n")
360     opcao = lerInteiro("O que deseja fazer?\n 1 - Atacar | 2 -
        Curar | 3 - Status \n > ")
361     limparTela()
362 }
363
364 se (opcao == 1) {
365     numRandom = u.sorteia(0, 2)
366     se (numRandom == 0) { digitar("Voc errou o ataque...\n") }
367     senao se (numRandom == 1) { digitar("Voc acertou o
        ataque!\n") }
368     senao { digitar("ATAQUE CRÍTICO!!!\n") }
369
370     statusInimigo[0] -= (statusPlayer[1] * numRandom)
371     limparTela()
372
373     se (statusInimigo[0] > 0) {
374         numRandom = u.sorteia(0, 2)
375         inteiro novoDanoInimigo = statusInimigo[1] * numRandom
376
377         se (numRandom == 0) {
378             digitar("O inimigo errou o ataque!\n")
379             limparTela()
380         }
381         senao {
382             digitar("O inimigo te causou ")
383             escreva(novoDanoInimigo)
384             digitar(" de dano!\n")
385             statusPlayer[0] -= novoDanoInimigo
386             limparTela()
387         }
388     } senao {
389         digitar("Voc derrotou ")
390         escreva(nomeInimigos[indiceInimigo])
391         digitar("!\n")
392         limparTela()
393     }
394 }
395 senao se (opcao == 2) {
396     se (statusPlayer[3] > 0) {
397         statusPlayer[0] += 30
398         statusPlayer[3] -= 1
399         digitar("Voc se curou! Restam " + statusPlayer[3] + "
            curas.\n")
```

```
400     limparTela()
401
402     se (statusPlayer[0] > vidaPlayerMax) {
403         statusPlayer[0] = vidaPlayerMax
404     }
405 } senao {
406     digitar("Sem poções de cura!\n")
407     limparTela()
408 }
409
410 inteiro danoExtra = statusInimigo[1] * u.sorteia(0, 1)
411 statusPlayer[0] -= danoExtra
412 digitar(nomeInimigos[indiceInimigo] + " te golpeou enquanto
413         voc se curava: -" + danoExtra + "\n")
414 limparTela()
415 }
416 senao se (opcao == 3) {
417     mostrarStatus()
418     limparTela()
419 }
420 digitar("\n --- FIM DO COMBATE --- \n")
421 limparTela()
422 }
423
424 funcao mostrarStatus() {
425     limpa()
426     escreva("Vida: " + statusPlayer[0] + " PV\n")
427     escreva("Ataque: " + statusPlayer[1] + " ATQ\n")
428     escreva("Velocidade: " + statusPlayer[2] + " VEL\n")
429     escreva("Quantidade de Curas: " + statusPlayer[3] + " C\n")
430     u.aguarde(1500)
431 }
432 funcao mostrarStatusAdmin() {
433     limpa()
434     escreva("Classe Guerreiro:\n")
435     escreva("Vida: " + "100" + " PV\n")
436     escreva("Ataque: " + "20" + " ATQ\n")
437     escreva("Velocidade: " + "5" + " VEL\n")
438     escreva("Quantidade de Curas: " + "5" + " C\n")
439
440     u.aguarde(400)
441
442     escreva("Classe Assassino:\n")
443     escreva("Vida: " + "50" + " PV\n")
444     escreva("Ataque: " + "25" + " ATQ\n")
445     escreva("Velocidade: " + "6" + " VEL\n")
```

```
446     escreva("Quantidade de Curas: " + "3" + " C\n")
447
448     u.aguarde(400)
449
450     escreva("Classe Tank:\n")
451     escreva("Vida: " + "120" + " PV\n")
452     escreva("Ataque: " + "18" + " ATQ\n")
453     escreva("Velocidade: " + "2" + " VEL\n")
454     escreva("Quantidade de Curas: " + "2" + " C\n")
455
456     u.aguarde(1500)
457 }
458
459
460 funcao mostrarStatusInimigosAdmin() {
461     limpa()
462     escreva("Inimigo Esqueleto:\n")
463     escreva("Vida: " + "50" + " PV\n")
464     escreva("Ataque: " + "15" + " ATQ\n")
465     escreva("Velocidade: " + "5" + " VEL\n")
466
467     u.aguarde(400)
468
469     escreva("Inimigo Globin:\n")
470     escreva("Vida: " + "60" + " PV\n")
471     escreva("Ataque: " + "15" + " ATQ\n")
472     escreva("Velocidade: " + "4" + " VEL\n")
473
474     u.aguarde(400)
475
476     escreva("Inimigo Dem nio:\n")
477     escreva("Vida: " + "60" + " PV\n")
478     escreva("Ataque: " + "20" + " ATQ\n")
479     escreva("Velocidade: " + "4" + " VEL\n")
480
481     u.aguarde(400)
482
483     escreva("Inimigo Cavaleiro Negro:\n")
484     escreva("Vida: " + "100" + " PV\n")
485     escreva("Ataque: " + "20" + " ATQ\n")
486     escreva("Velocidade: " + "6" + " VEL\n")
487
488     u.aguarde(400)
489
490     escreva("Inimigo Rei Dem nio:\n")
491     escreva("Vida: " + "130" + " PV\n")
492     escreva("Ataque: " + "23" + " ATQ\n")
```



```

538     limpa()
539     para(inteiro i = 0; i < 3; i++) {
540         escreva("* * * * * * * * * * *\n")
541         escreva("* * * * * * * * * * *\n")
542         escreva("*   * 2 * 3 * 4 * 5 *\n")
543         u.aguarde(450)
544         limpa()
545         escreva("* * * * * * * * * * *\n")
546         escreva("* * * * * * * * * * *\n")
547         escreva("* 1 * 2 * 3 * 4 * 5 *\n")
548         u.aguarde(450)
549         limpa()
550     }
551     limpa()
552     digitarLento("Sala 1           \n")
553     u.aguarde(450)
554     digitarLento("Entrando na caverna... \n")
555     pare
556 caso 2:
557     limpa()
558     para(inteiro i = 0; i < 3; i++) {
559         escreva("* * * * * * * * * * *\n")
560         escreva("* * * * * * * * * * *\n")
561         escreva("* 1 *   * 3 * 4 * 5 *\n")
562         u.aguarde(450)
563         limpa()
564         escreva("* * * * * * * * * * *\n")
565         escreva("* * * * * * * * * * *\n")
566         escreva("* 1 * 2 * 3 * 4 * 5 *\n")
567         u.aguarde(450)
568         limpa()
569     }
570     limpa()
571     digitarLento("Sala 2           \n")
572     u.aguarde(450)
573     digitarLento("Goblins a espreita...   \n")
574     pare
575 caso 3:
576     limpa()
577     para(inteiro i = 0; i < 3; i++) {
578         escreva("* * * * * * * * * * *\n")
579         escreva("* * * * * * * * * * *\n")
580         escreva("* 1 * 2 *   * 4 * 5 *\n")
581         u.aguarde(450)
582         limpa()
583         escreva("* * * * * * * * * * *\n")
584         escreva("* * * * * * * * * * *\n")

```

```

585     escreva("* 1 * 2 * 3 * 4 * 5 *\n")
586     u.aguarde(450)
587     limpa()
588 }
589 limpa()
590 digitarLento("Sala 3          \n")
591 u.aguarde(450)
592 digitarLento("Uma água benta cairia bem...    \n")
593 pare
594 caso 4:
595     limpa()
596     para(inteiro i = 0; i < 3; i++) {
597         escreva("* * * * * *\n")
598         escreva("* * * * * *\n")
599         escreva("* 1 * 2 * 3 *   * 5 *\n")
600         u.aguarde(450)
601         limpa()
602         escreva("* * * * * *\n")
603         escreva("* * * * * *\n")
604         escreva("* 1 * 2 * 3 * 4 * 5 *\n")
605         u.aguarde(450)
606         limpa()
607     }
608     limpa()
609     digitarLento("Sala 4          \n")
610     u.aguarde(450)
611     digitarLento("Lar do melhor espadachim do submundo...    \n")
612     u.aguarde(600)
613     pare
614 caso 5:
615     limpa()
616     para(inteiro i = 0; i < 3; i++) {
617         escreva("* * * * * *\n")
618         escreva("* * * * * *\n")
619         escreva("* 1 * 2 * 3 * 4 *   *\n")
620         u.aguarde(450)
621         limpa()
622         escreva("* * * * * *\n")
623         escreva("* * * * * *\n")
624         escreva("* 1 * 2 * 3 * 4 * 5 *\n")
625         u.aguarde(450)
626         limpa()
627     }
628     limpa()
629     digitarLento("Sala 5 FINAL          \n")
630     u.aguarde(600)
631     digitarLento("É HORA DO EXPURGO FINAL!!!    \n")

```

```
632         u.aguarde(600)
633         pare
634     }
635 }
636 }
```

Listing 3.2 – Fim do Código em Portugol



## 4 Código Fonte

### 4.1 Versão Inicial

```

1  /*
2   Nome do Projeto : RPG Texto Bloodivine
3
4   Nome dos Integrantes do Grupo :
5   Nome : Gustavo Silva Faustino
6   Nome : Danniel Angelo Araujo Fonseca
7   Nome : Lucas Calil Braga Gomes
8   Nome : Kevin da Silva Lima
9   Nome : K a u Oliveira dos Santos
10  Nome : Valmir Martins Dos Santos Junior
11  Nome : Arthur Vinicius Santos
12
13  Data de criação do projeto : 12/02/2026
14  Data de Entrega do Projeto : 01/05/2026
15
16  */
17  import java.util.Scanner;
18  public class Main6 {
19
20      public static void digitar(String texto) throws
21      InterruptedException {
22          for (int i = 0; i < texto.length(); i++) {
23              System.out.print(texto.charAt(i));
24              Thread.sleep(20);
25          }
26          System.out.println();
27
28      public static Player escolhaClasse() throws
29      InterruptedException {
30          Scanner sc = new Scanner(System.in);
31          Player player = Player.criarPlayer(rpgClasses.GUERREIRO);
32
33          digitar("Sua história no RPG de " + cores.vermelhoForte
34              + "BLOODVINE" + " INICIOU!!!" + cores.reset);

```

```
34     Thread.sleep(1500);
35     limparTela();
36
37     while (true) {
38
39         digitar("Digite 1, 2 ou 3 para escolher sua classe
40                 no RPG de " + cores.vermelhoForte + "BLOODVINE" +
41                 cores.reset);
42         System.out.println("1)          " +
43                             rpgClasses.GUERREIRO);
44         System.out.println("2)          " +
45                             rpgClasses.ASSASSINO);
46         System.out.println("3)          " + rpgClasses.TANK);
47
48         if (!sc.hasNextInt()) {
49             limparTela();
50             digitar(" Escolha Inv lida. Digite um n mero
51                     de 1 a 3.");
52             sc.next();
53             Thread.sleep(1000);
54             limparTela();
55             continue;
56         }
57
58         int numClasse = sc.nextInt();
59
60         if (numClasse == 1) {
61             player =
62                 Player.criarPlayer(rpgClasses.GUERREIRO);
63             digitar("Sua escolha foi a classe " +
64                     rpgClasses.GUERREIRO);
65             Thread.sleep(2000);
66             limparTela();
67             return player;
68
69         } else if (numClasse == 2) {
70             player =
71                 Player.criarPlayer(rpgClasses.ASSASSINO);
72             digitar("Sua escolha foi a classe " +
73                     rpgClasses.ASSASSINO);
74             Thread.sleep(2000);
75             limparTela();
```

```
67         return player;
68
69     } else if (numClasse == 3) {
70         player = Player.criarPlayer(rpgClasses.TANK);
71         digitar("Sua escolha foi a classe " +
72             rpgClasses.TANK);
73         Thread.sleep(2000);
74         limparTela();
75         return player;
76     }
77
78     if (numClasse > 3 || numClasse < 1) {
79         limparTela();
80         digitar(" Escolha Inv lida. Digite um n mero
81             de 1 a 3.");
82         Thread.sleep(1000);
83         limparTela();
84     }
85 }
86
87 public static void mostrarMenu(Inimigo inimigo, Player
88     player, int quantidadeCura, int round) {
89     System.out.println("Round: " + round);
90     System.out.println("=====");
91     System.out.println(cores.verde + " Sua Vida: " +
92         player.vida + "/" + player.vidaMax + cores.reset);
93     System.out.println(cores.vermelho + " Vida " +
94         inimigo.nome + ": " + inimigo.vida + cores.reset);
95     System.out.println(cores.azul + " Curas
96         disponiveis: " + quantidadeCura + "\033[0m" +
97         cores.reset);
98     System.out.println("=====");
99     System.out.println("Escolha sua a o:");
100     System.out.println("1) Atacar");
101     System.out.println("2) Curar");
102     System.out.println("3) Status");
103     System.out.print(" Op o:");
104 }
105
106 public static void mostrarMenu2(Inimigo inimigo, int
107     quantidadeCura, int round, Player player) {
```

```
101     System.out.println("Round: " + round);
102     System.out.println("=====");
103     System.out.println(cores.verde + "          Sua Vida: " +
104         player.vida + "/" + player.vidaMax + cores.vermelho +
105         " -" + inimigo.dano + cores.reset);
106     System.out.println(cores.vermelho + "          Vida " +
107         inimigo.nome + ": " + inimigo.vida + cores.reset);
108     System.out.println(cores.azul + "          Curas
109         disponiveis: " + quantidadeCura + "\033[0m" +
110         cores.reset);
111     System.out.println("=====");
112     System.out.println("Escolha sua acao:");
113     System.out.println("1)          Atacar");
114     System.out.println("2)          Curar");
115     System.out.println("3)          Status");
116     System.out.print("Opcao:");
117 }
118
119 public static void limparTela() {
120     for (int i = 0; i < 50; i++) System.out.println();
121 }
122
123 public static class cores {
124     static final String verde = "\033[32m";
125     static final String vermelho = "\033[31m";
126     static final String azul = "\033[34m";
127     static final String roxo = "\033[35m";
128     static final String amareloForte = "\033[1;33m";
129     static final String cianoForte = "\033[1;36m";
130     static final String vermelhoForte = "\033[1;31m";
131     static final String negrito = "\033[1m";
132     static final String reset = "\033[0m";
133 }
134
135 public static class Player {
136     int vida;
137     int vidaMax;
138     int dano;
139     int cura;
140     int quantidadeCura;
141     String classe;
```

```
138     public void atacar(Inimigo inimigo) {
139         inimigo.vida -= dano;
140     }
141
142     public void curar() {
143         if (quantidadeCura > 0) {
144             vida += cura;
145
146             if (vida > vidaMax)
147                 vida = vidaMax;
148
149             quantidadeCura--;
150
151             System.out.println("      Voc se curou! Vida
152                 atual: " + cores.verde + vida + "/" + vidaMax
153                 + cores.reset);
154         } else {
155             System.out.println(cores.amareloForte +
156                 "      Voc n o tem mais curas dispon veis!" +
157                 cores.reset);
158         }
159     }
160
161     public void receberDano(int dano) {
162         vida -= dano;
163     }
164
165     public static Player criarPlayer(String classeEscolhida)
166     {
167
168         if (classeEscolhida.equals(rpgClasses.GUERREIRO)) {
169             Player p = new Player();
170             p.vida = 100;
171             p.vidaMax = 100;
172             p.dano = 20;
173             p.cura = 30;
174             p.quantidadeCura = 4;
175             p.classe = rpgClasses.GUERREIRO;
176             return p;
177
178         } else if
179             (classeEscolhida.equals(rpgClasses.ASSASSINO)) {
```

```
174         Player p = new Player();
175         p.vida = 90;
176         p.vidaMax = 90;
177         p.dano = 15;
178         p.cura = 30;
179         p.quantidadeCura = 3;
180         p.classe = rpgClasses.ASSASSINO;
181         return p;
182
183     } else if (classeEscolhida.equals(rpgClasses.TANK)) {
184         Player p = new Player();
185         p.vida = 150;
186         p.vidaMax = 150;
187         p.dano = 15;
188         p.cura = 40;
189         p.quantidadeCura = 5;
190         p.classe = rpgClasses.TANK;
191         return p;
192     }
193
194     return null;
195 }
196
197
198 public static class rpgClasses {
199     public static final String GUERREIRO = cores.negrito +
200         cores.vermelhoForte + "GUERREIRO" + cores.reset;
201     public static final String ASSASSINO = cores.negrito +
202         cores.roxo + "ASSASSINO" + cores.reset;
203     public static final String TANK = cores.negrito +
204         cores.azul + "TANK" + cores.reset;
205 }
206
207 public static class Inimigos {
208     public static final String ESQUELETO = "ESQUELETO";
209     public static final String GOBLIN = "GOBLIN";
210 }
211
212 public static class Inimigo {
213     int vida;
214     int dano;
215     String nome;
```

```
213
214     public void atacar(Player player) {
215         player.vida -= dano;
216     }
217
218     public static Inimigo gerarInimigo(String nomeInimigo) {
219
220         if (nomeInimigo.equals(Inimigos.ESQUELETO)) {
221             Inimigo i = new Inimigo();
222             i.vida = 60;
223             i.dano = 15;
224             i.nome = Inimigos.ESQUELETO;
225             return i;
226
227         } else if (nomeInimigo.equals(Inimigos.GOBLIN)) {
228             Inimigo i = new Inimigo();
229             i.vida = 50;
230             i.dano = 10;
231             i.nome = Inimigos.GOBLIN;
232             return i;
233         }
234
235         return null;
236     }
237 }
238
239 public static void combate(Player player, Inimigo inimigo,
240     Scanner sc) throws InterruptedException {
241
242     int round = 1;
243     boolean tomouDano = false;
244     int danoAcumulado = 0;
245
246     digitar("Apareceu um " + inimigo.nome + "!!");
247
248     while (player.vida > 0 && inimigo.vida > 0) {
249
250         if (tomouDano) {
251             mostrarMenu2(inimigo, player.quantidadeCura,
                round, player);
252         } else {
```

```
252         mostrarMenu(inimigo, player,
253                     player.quantidadeCura, round);
254     }
255
256     if (!sc.hasNextInt()) {
257         limparTela();
258         digitar(" Escolha Inv lida. Digite um n mero
259                 de 1 a 3.");
260         sc.next();
261         Thread.sleep(1000);
262         limparTela();
263         continue;
264     }
265
266     int escolha = sc.nextInt();
267
268     switch (escolha) {
269
270     case 1:
271         player.atacar(inimigo);
272         inimigo.atacar(player);
273         danoAcumulado += inimigo.dano;
274         tomouDano = true;
275
276         digitar(" Voc atacou o " +
277                 inimigo.nome + "!");
278         digitar(" 0 " + inimigo.nome + "
279                 contra-atacou!");
280         digitar(" Dano recebido:" +
281                 cores.vermelho + " -" + inimigo.dano +
282                 cores.reset);
283
284         Thread.sleep(2500);
285         round++;
286         limparTela();
287         break;
288
289     case 2:
290         player.curar();
291         inimigo.atacar(player);
292         danoAcumulado += inimigo.dano;
293         tomouDano = true;
```

```
288
289         digitar("    0    " + inimigo.nome + " te
290             atacou!");
291     digitar("    Dano    recebido: " +
292         cores.vermelho + " -" + inimigo.dano +
293         cores.reset);
294
295     Thread.sleep(2500);
296     round++;
297     limparTela();
298     break;
299
300 case 3:
301     if (tomouDano) {
302         limparTela();
303         System.out.println("====Status====");
304         System.out.println("Dano b sico: " +
305             cores.vermelhoForte + player.dano +
306             cores.reset);
307         System.out.println("Vida: " +
308             cores.verde + player.vida + "/" +
309             player.vidaMax + cores.reset);
310         System.out.println("Total dano recebido:
311             " + cores.vermelho + "-" +
312             danoAcumulado + cores.reset);
313         Thread.sleep(3000);
314     } else {
315         limparTela();
316         System.out.println("====Status====");
317         System.out.println("Dano b sico: " +
318             cores.vermelhoForte + player.dano +
319             cores.reset);
320         System.out.println("Vida: " +
321             cores.verde + player.vida + "/" +
322             player.vidaMax + cores.reset);
323         Thread.sleep(3000);
324         limparTela();
325     }
326     break;
```

```
317         limparTela();
318     }
319
320     if (player.vida <= 0) {
321         digitar("      Voc      morreu");
322     } else {
323         System.out.println("=====");
324         digitar("=====      Voc      venceu=====");
325     }
326 }
327
328 public static void main(String[] args) throws
    InterruptedException {
329
330     Scanner sc = new Scanner(System.in);
331
332     Player player = escolhaClasse();
333     Inimigo inimigo =
        Inimigo.gerarInimigo(Inimigos.ESQUELETO);
334
335     digitar("              " + cores.negrito +
        cores.vermelhoForte + "BLOODIVINE " +
        cores.amareloForte + "INICIANDO... " +
        cores.cianoForte + "Prepare-se para a batalha!
        " + cores.reset);
336
337     Thread.sleep(3000);
338
339     combate(player, inimigo, sc);
340
341     sc.close();
342 }
343 }
```

Listing 4.1 – Exemplo de classe Java

# Conceitos Utilizados no Código do Projeto Não Apresentados em Aula

---

Durante o desenvolvimento do projeto, foram utilizados alguns recursos da linguagem Java que não haviam sido abordados em aula, mas que foram importantes para melhorar a organização do código e a experiência do usuário no jogo.

## Uso do `Thread.sleep()`

---

O comando `Thread.sleep()` é utilizado para pausar a execução do programa por um determinado tempo. No projeto, ele foi usado para criar pequenos intervalos entre mensagens exibidas no terminal, deixando a interação mais dinâmica e semelhante a um jogo.

**Exemplo utilizado no código:**

```
Thread.sleep(1500);
```

Nesse caso, o programa aguarda 1500 milissegundos (1,5 segundos) antes de continuar sua execução.

O recurso também foi utilizado no método `digitar()` para criar um efeito de digitação lenta:

```
for (int i = 0; i < texto.length(); i++) {  
    System.out.print(texto.charAt(i));  
    Thread.sleep(20);  
}
```

Assim, cada caractere é exibido com uma pequena pausa, simulando uma animação de texto.

## Uso da `InterruptedException` e do comando `throws`

---

Sempre que o `Thread.sleep()` é utilizado, o Java exige que o programa trate a possibilidade de interrupção da pausa durante a execução. Para isso, foi utilizada a exceção `InterruptedException`.

No código, isso aparece junto do comando `throws`:

```
public static void digitar(String texto) throws InterruptedException
```

O comando `throws` informa que aquela função pode gerar uma exceção durante sua execução. Nesse caso, ele avisa que o `Thread.sleep()` pode causar uma `InterruptedException`.

Sem essa declaração, o código não seria compilado corretamente pelo Java.

## Uso de cores no terminal com códigos ANSI

---

O projeto utiliza códigos ANSI para alterar as cores dos textos exibidos no terminal. Isso foi utilizado para destacar informações importantes do jogo, como vida do jogador, dano recebido e nomes das classes.

As cores foram organizadas dentro da classe `cores`:

```
public static class cores {  
    static final String verde = "\033[32m";  
    static final String vermelho = "\033[31m";  
    static final String reset = "\033[0m";  
}
```

Esses códigos especiais são interpretados pelo terminal e alteram a cor do texto exibido.

**Exemplo de uso:**

```
System.out.println(cores.verde + "❤️ Sua Vida: " + player.vida +  
cores.reset);
```

Nesse exemplo, o texto será exibido na cor verde até o uso do `reset`, que retorna à cor padrão do terminal.

## Uso do `static`

---

A palavra-chave `static` foi utilizada para permitir que algumas funções e variáveis sejam acessadas diretamente pelo nome da classe, sem a necessidade de criar um objeto.

### Exemplo:

```
public static void limparTela()
```

Essa função pode ser utilizada diretamente dentro do programa:

```
limparTela();
```

O `static` também foi utilizado para criar valores fixos, como as cores e os nomes das classes do jogo.

## Uso das classes `Player` e `Inimigo`

---

No projeto, as classes `Player` e `Inimigo` foram utilizadas para organizar os dados e funções relacionadas aos personagens do jogo.

Essas classes funcionam como estruturas separadas dentro do código, contendo informações próprias, como vida, dano e nome, além de funções específicas para ataque e cura.

### Exemplo da classe `Player` :

```
public static class Player {  
    int vida;  
    int dano;  
    String classe;  
}
```

**Exemplo da classe Inimigo :**

```
public static class Inimigo {  
    int vida;  
    int dano;  
    String nome;  
}
```

No programa principal, essas classes são utilizadas para criar os personagens do jogo:

```
Player player = escolhaClasse();  
Inimigo inimigo = Inimigo.gerarInimigo(Inimigos.ESQUELETO);
```

Dessa forma, o código fica mais organizado, separando as funções e informações relacionadas a cada parte do jogo.

# Resumo dos Erros do Código Java - RPG BLOODVINE

---

Os problemas que identificamos no código envolvem validação de entrada, lógica de combate, tratamento de exceções e erros de sintaxe que causam travamentos, loops infinitos ou falhas no programa.

| Erro                            | Descrição                                                                                                               |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Erro no if vazio                | O trecho <code>if ()</code> está incorreto, pois o Java exige uma condição dentro dos parênteses.                       |
| Problema de input inválido      | O programa pode quebrar se o usuário digitar letras em vez de números, devido à falta de validação.                     |
| Loop infinito no Scanner        | A ausência de <code>sc.next()</code> faz com que o Scanner continue lendo o mesmo valor inválido repetidamente.         |
| Ausência do continue            | Sem a instrução <code>continue</code> , o programa pode gerar uma exceção do tipo <code>InputMismatchException</code> . |
| Vida negativa                   | O código não possui uma trava para impedir que os pontos de vida fiquem abaixo de zero.                                 |
| Loop infinito no combate        | Se o cálculo de dano não for aplicado corretamente, o combate pode nunca ser finalizado.                                |
| Uso excessivo de if e else      | A estrutura com muitos condicionais aninhados dificulta a manutenção do sistema.                                        |
| Travamento com Thread.sleep     | O uso de valores muito altos nesta função pode fazer com que o jogo pareça estar congelado.                             |
| Falta de tratamento de exceções | O código não utiliza blocos <code>try/catch</code> para gerenciar erros e falhas durante a execução.                    |
| Erro de sintaxe (return)        | Um erro de sintaxe na linha 177, onde há um <code>return</code> fora de contexto.                                       |

Main6.java > Main6 > Player > criarPlayer(String)

```
public class Main6 {
    public static class Player {
        public static Player criarPlayer(String classeEscolhida) {
            } else if (classeEscolhida.equals(rpgClasses.TANK)) {
                Player p = new Player();
                p.vida = 150;
                p.vidaMax = 150;
                p.dano = 15;
                p.cura = 40;
                p.quantidadeCura = 5;
                p.classe = rpgClasses.TANK;
                return p;
            }
        }
    }

    return ;
}

public static class rpgClasses {
```

```
public class Main6 {
    public static Player escolhaClasse() throws InterruptedException {
        if (numClasse == 1) {
            player = Player.criarPlayer(rpgClasses.GUERREIRO);
            digitar("Sua escolha foi a classe " + rpgClasses.GUERREIRO);
            Thread.sleep(millis: 2000);
            limparTela();
            return ;
        } else if (numClasse == 2) {
            player = Player.criarPlayer(rpgClasses.ASSASSINO);
            digitar("Sua escolha foi a classe " + rpgClasses.ASSASSINO);
            Thread.sleep(millis: 2000);
            limparTela();
            return ;
        } else if (numClasse == 3) {
            player = Player.criarPlayer(rpgClasses.TANK);
            digitar("Sua escolha foi a classe " + rpgClasses.TANK);
            Thread.sleep(millis: 2000);
            limparTela();
            return ;
        }
    }
}
```

```
aa java U J Main6.java 1, U X
J Main6.java > Main6 > mostrarMenu2(Inimigo inimigo, int, int, Player)
3 public class Main6 {
71 public static void mostrarMenu(Inimigo inimigo, Player player, int quantidadeCura, int round) {
72     System.out.println(x: "=====");
73     System.out.println(cores.verde + "♥ Sua Vida: " + player.vida + "/" + player.vidaMax + cores.reset);
74     System.out.println(cores.vermelho + "💀 Vida " + inimigo.nome + ": " + inimigo.vida + cores.reset);
75     System.out.println(cores.azul + "💊 Curas disponiveis: " + quantidadeCura + "\033[0m" + cores.reset);
76     System.out.println(x: "=====");
77     System.out.println(x: "Escolha sua ação:");
78     System.out.println(x: "1) ⚔️ Atacar");
79     System.out.println(x: "2) 🩹 Curar");
80     System.out.print(s: "Opção:");
81 }
82
83
84 public static void mostrarMenu2(Inimigo inimigo, int quantidadeCura, int round, Player player) {
85     System.out.println("Round: " + round);
86     System.out.println(x: "=====");
87     System.out.println(cores.verde + "♥ Sua Vida: " + player.vida + "/" + player.vidaMax + cores.vermelho + " -" + inimigo.dano + cores.reset);
88     System.out.println(cores.vermelho + "💀 Vida " + inimigo.nome + ": " + inimigo.vida + cores.reset);
89     System.out.println(cores.azul + "💊 Curas disponiveis: " + quantidadeCura + "\033[0m" + cores.reset);
90     System.out.println(x: "=====");
91     System.out.println(x: "Escolha sua ação:");
92     System.out.println(x: "2) 🩹 Curar");
93     System.out.println(x: "3) 🏠 Status");
94     System.out.print(s: "Opção:");
95 }
96
97 public static void limparTela() {
```

```
if () {  
    limparTela();  
    digitar(texto: "❌ Escolha Inválida. Digite um número de 1 a 3.");  
    sc.next();  
    Thread.sleep(millis: 1000);  
    limparTela();  
    continue;  
}
```

## 4.2 Versão Final



```
// ||
// ||

public class Main7 {

    // Globais do Random e Scanner
    private static final Random rand = new Random();
    private static final Scanner scan = new Scanner(System.in);

    // [MAIN]
    public static void main(String[] args) throws InterruptedException {
        Player player = new Player();
        Inimigo inimigo = new Inimigo();
        while(true) {
            System.out.println("1 - Jogar Normal");
            System.out.println("2 - Modo Admin");
            System.out.println("3 - Sair");

            int modo = lerOpcaoValida(1, 3);
            if (modo == 2) {
                if(admin()) {
                    System.out.println("✅ Você entrou no modo admin");
                    Thread.sleep(1500);
                } else {
                    System.out.println("❌ Senha incorreta");
                    return;
                }
                player = criarPlayerAdmin();
                escolherinimigo(player, inimigo);
                return;
            }
            else if (modo == 1) {
                jogoNormal(player);
                break;
            }
            else {
                break;
            }
        }
    }

    // [ADMIN]

    public static boolean admin(){
        System.out.println("Digite sua senha");
    }
}
```

```

        String senha = scan.next();

        return senha.equals("admin12");
    }

    public static Player criarPlayerAdmin() throws InterruptedException {

        Player player = null;

        while (true) {
            limparTela();
            System.out.println("==== MODO ADMIN =====");
            System.out.println("Escolha sua classe");
            System.out.println(Cores.NEGRITO + Cores.VERMELHO_FORTE + "1) GUERREIRO" + Cores.RESET);
            System.out.println(Cores.NEGRITO + Cores.ROXO + "2) ASSASSINO" + Cores.RESET);
            System.out.println(Cores.NEGRITO + Cores.AZUL + "3) TANK" + Cores.RESET);

            int escolha = lerOpcaoValida(1, 3);
            switch (escolha) {
                case 1:
                    player = Player.criar(ClassessRpg.GUERREIRO);break;
                case 2:
                    player = Player.criar(ClassessRpg.ASSASSINO);break;
                case 3:
                    player = Player.criar(ClassessRpg.TANK);break;
                default:
                    System.out.println("Classe invalida!");
                    return Player.criar(ClassessRpg.GUERREIRO); // Define automaticamente como GUERREIRO
            }
            break;
        }
        limparTela();

        System.out.println("=== EDITAR STATUS ===");

        System.out.println("Vida: ");
        player.vida = lerOpcaoValida(1, 999);
        player.vidaMax = player.vida;

        System.out.println("Dano: ");
        player.dano = lerOpcaoValida(1, 999);
    }

```

```

        System.out.println("Quantidade de Cura: ");
        player.quantidadeCura = lerOpcaoValida(1, 999);
        limparTela();

        System.out.println(Cores.AZUL + "SUA CLASSE FOI CRIADA" +
Cores.RESET);

        System.out.println(Cores.VERDE + "Vida: " + player.vida +
Cores.RESET);
        System.out.println(Cores.VERMELHO + "Dano: " + player.dano +
Cores.RESET);
        System.out.println(Cores.AZUL + "Cura: " + player.cura +
Cores.RESET);
        limparTela();
        return player;
    }

    public static void escolherinimigo(Player player, Inimigo inimigo)
throws InterruptedException{

        while (true) {
            System.out.println("Escolha o inimigo:");
            System.out.println(Cores.VERMELHO + "1) ESQUELETO" +
Cores.RESET);
            System.out.println(Cores.VERMELHO + "2) GOBLIN" + Cores.RESET);
            System.out.println(Cores.VERMELHO + "3) DEMONIO" + Cores.RESET);
            System.out.println(Cores.VERMELHO + "4) CAVALEIRO NEGRO" +
Cores.RESET);
            System.out.println(Cores.VERMELHO_FORTE + "5) REI DEMONIO" +
Cores.RESET);
            System.out.println(Cores.NEGRITO + "6) SAIR" + Cores.RESET);

            int escolha = lerOpcaoValida(1, 6);
            switch (escolha) {
                case 1:
                    inimigo = Inimigo.criar(Inimigos.ESQUELETO);
                    executarFase(player, inimigo, escolha, escolha);
                    break;
                case 2:
                    inimigo = Inimigo.criar(Inimigos.GOBLIN);
                    executarFase(player, inimigo, escolha, escolha);
                    break;
                case 3:
                    inimigo = Inimigo.criar(Inimigos.DEMONIO);
                    executarFase(player, inimigo, escolha, escolha);

```

```

        break;
    case 4:
        inimigo = Inimigo.criar(Inimigos.CAVALIRO_NEGRO);
        executarFase(player, inimigo, escolha, escolha);
        break;
    case 5:
        inimigo = Inimigo.criar(Inimigos.REI_DEMONIO);
        executarFase(player, inimigo, escolha, escolha);
        break;
    default:
        limparTela();
        return;
    }
    limparTela();
}
}

public static void jogoNormal(Player player) throws InterruptedException
{
    player = escolhaClasse();
    Inimigo esqueleto = Inimigo.criar(Inimigos.ESQUELETO);
    Inimigo goblin = Inimigo.criar(Inimigos.GOBLIN);
    Inimigo demonio = Inimigo.criar(Inimigos.DEMONIO);
    Inimigo cavaleiroNegro = Inimigo.criar(Inimigos.CAVALIRO_NEGRO);
    Inimigo reiDemonio = Inimigo.criar(Inimigos.REI_DEMONIO);

    digitar("⚔️ " + Cores.NEGRITO + Cores.VERMELHO_FORTE + "BLOODIVINE "
        + Cores.AMARELO_FORTE + "INICIANDO... "
        + Cores.CIANO_FORTE + "Prepare-se para a batalha! ⚔️" +
Cores.RESET);
    Thread.sleep(1500);
    introducao();
    if (!executarFase(player, esqueleto, 1, 1)) return;
    pos_batalha_Esqueleto();
    if (!executarFase(player, goblin, 2, 2)) return;
    pos_batalha_Goblin();
    if (!executarFase(player, demonio, 3, 3)) return;
    pos_batalha_Demonio();
    if (!executarFase(player, cavaleiroNegro, 4, 4)) return;
    pos_batalha_Cavaleiro_Negro();
    if (!executarFase(player, reiDemonio, 5, 5)) return;

    limparTela();
    finalDoJogo();
}

```

```

        scan.close();
    }

    // [UTILITÁRIOS]

    // [UTILITÁRIOS > Console]

    public static void digitar(String texto) throws InterruptedException {
        for (char c : texto.toCharArray()) {
            System.out.print(c);
            Thread.sleep(20);
        }
        System.out.println();
    }

    public static void limparTela() {
        for (int i = 0; i < 50; i++) System.out.println();
    }

    public static void limparTelaDevagar() throws InterruptedException {
        for (int i = 0; i < 50; i++) {System.out.println();
Thread.sleep(40);}
    }

    public static int lerOpcaoValida(int min, int max) throws
InterruptedException {
        while (true) {
            if (!scan.hasNextInt()) {

                digitar("❌ Opção inválida. Digite um número de " + min + "
a " + max + ".");
                scan.next();
                Thread.sleep(1000);

                continue;
            }
            int opcao = scan.nextInt();
            if (opcao >= min && opcao <= max) return opcao;

            digitar("❌ Opção inválida. Digite um número de " + min + " a "
+ max + ".");
            Thread.sleep(1000);

        }
    }

```

```

}

// [UTILITÁRIOS > Cores]

public static class Cores {
    public static final String VERDE           = "\033[32m";
    public static final String VERDE_BRILHANTE = "\033[1;32m";
    public static final String VERMELHO       = "\033[31m";
    public static final String AZUL           = "\033[34m";
    public static final String AZUL_BRILHANTE = "\033[1;34m";
    public static final String ROXO           = "\033[35m";
    public static final String AMARELO_FORTE  = "\033[1;33m";
    public static final String CIANO_FORTE    = "\033[1;36m";
    public static final String VERMELHO_FORTE = "\033[1;31m";
    public static final String NEGRITO        = "\033[1m";
    public static final String RESET          = "\033[0m";
}

// [CLASSES RPG]
public static class ClassesRpg {
    public static final String GUERREIRO = "GUERREIRO";
    public static final String ASSASSINO = "ASSASSINO";
    public static final String TANK      = "TANK";

    // Separei a string colorida da string normal para poder usar EX:
    Player.criar(ClassesRpg.GUERREIRO)
    // Esse método aqui mostra a string colorida
    public static String formatado(String classe) {
        switch (classe) {
            case GUERREIRO: return Cores.NEGRITO + Cores.VERMELHO_FORTE
+ "GUERREIRO" + Cores.RESET;
            case ASSASSINO: return Cores.NEGRITO + Cores.ROXO
+ "ASSASSINO" + Cores.RESET;
            case TANK:      return Cores.NEGRITO + Cores.AZUL
+ "TANK"      + Cores.RESET;
            default:        return classe;
        }
    }
}

// [INIMIGOS]

// [INIMIGOS > Constantes]

public static class Inimigos {

```

```

        public static final String ESQUELETO        = "ESQUELETO";
        public static final String GOBLIN           = "GOBLIN";
        public static final String DEMONIO          = "DEMÔNIO";
        public static final String CAVALEIRO_NEGRO  = "CAVALEIRO-NEGRO";
        public static final String REI_DEMONIO      = "REI-DEMÔNIO";
    }

    // [INIMIGOS > Modelo]

    public static class Inimigo {
        int vida;
        int dano;
        String nome;

        public void atacar(Player player) {
            player.receberDano(dano);
        }

        public static Inimigo criar(String nome) {
            Inimigo ini = new Inimigo();
            ini.nome = nome;

            switch (nome) {
                case Inimigos.ESQUELETO:
                    ini.vida = 50; ini.dano = rand.nextInt(6) + 5; break;
                case Inimigos.GOBLIN:
                    ini.vida = 80; ini.dano = rand.nextInt(10) + 8; break;
                case Inimigos.DEMONIO:
                    ini.vida = 100; ini.dano = rand.nextInt(15) + 12; break;
                case Inimigos.CAVALERO_NEGRO:
                    ini.vida = 130; ini.dano = rand.nextInt(16) + 15; break;
                case Inimigos.REI_DEMONIO:
                    ini.vida = 220; ini.dano = rand.nextInt(20) + 20; break;
                default:
                    throw new IllegalArgumentException("Inimigo
desconhecido: " + nome);
            }
            return ini;
        }
    }

    // [PLAYER]

    // [PLAYER > Modelo]

```

```

public static class Player {
    int vida;
    int vidaMax;
    int dano;
    int cura;
    int quantidadeCura;
    String classe;

    public void receberDano(int dano) {
        this.vida -= dano;
    }

    public void curar() {
        if (quantidadeCura > 0) {
            this.vida = Math.min(this.vidaMax, this.vida + this.cura);
            this.quantidadeCura--;
        }
    }

    public void resetarVida() {
        this.vida = this.vidaMax;
    }

    public static Player criar(String classe) {
        Player p = new Player();
        p.classe = classe;

        switch (classe) {
            case ClassesRpg.GUERREIRO:
                p.vida = 100; p.vidaMax = 100; p.dano = 15; p.cura = 20;
                p.quantidadeCura = 3; break;
            case ClassesRpg.ASSASSINO:
                p.vida = 70; p.vidaMax = 70; p.dano = 25; p.cura = 15;
                p.quantidadeCura = 2; break;
            case ClassesRpg.TANK:
                p.vida = 150; p.vidaMax = 150; p.dano = 10; p.cura = 30;
                p.quantidadeCura = 4; break;
        }
        return p;
    }
}

// [COMBATE]

// [COMBATE > Iniciativa]

```

```

private static boolean playerTemIniciativa(Player player) {
    return rand.nextBoolean();
}

// [COMBATE > Crítico]

private static int calcularDanoCritico(int danoBase) {
    if (rand.nextInt(100) < 20) {
        return (int) (danoBase * 1.5);
    }
    return danoBase;
}

// [COMBATE > Ataques]

private static void aplicarAtaquePlayer(Player player, Inimigo inimigo)
throws InterruptedException {
    int dano = calcularDanoCritico(player.dano);
    inimigo.vida -= dano;
    digitar("🔪 Você atacou " + Cores.VERMELHO + inimigo.nome +
Cores.RESET
        + "! Dano: " + Cores.AZUL + dano + Cores.RESET);
}

// podeDarCritico = false quando o player está se curando
// Inimigo não pode acertar um critico durante a cura
private static void aplicarAtaqueInimigo(Inimigo inimigo, Player player,
boolean podeDarCritico) throws InterruptedException {
    int dano = 0;
    if (podeDarCritico)
        dano = calcularDanoCritico(inimigo.dano);
    else
        dano = inimigo.dano;
    player.receberDano(dano);
    digitar(Cores.VERMELHO + " " + inimigo.nome + " causou " + dano + "
de dano!" + Cores.RESET);
}

// [COMBATE > Turnos]

// Turno de ataque: rola iniciativa e resolve a ordem das ações.
private static void turnoAtaque(Player player, Inimigo inimigo) throws
InterruptedException {
    if (playerTemIniciativa(player)) {
        digitar(Cores.CIANO_FORTE + "⚡ Você age primeiro!" +

```

```

Cores.RESET);
    aplicarAtaquePlayer(player, inimigo);
    if (inimigo.vida > 0) {
        digitar(Cores.VERMELHO + inimigo.nome + " contra-ataca!" +
Cores.RESET);
        aplicarAtaqueInimigo(inimigo, player, true); // crítico
permitido
    }
    } else {
        digitar(Cores.VERMELHO + "⚡ " + inimigo.nome + " age primeiro!"
+ Cores.RESET);
        aplicarAtaqueInimigo(inimigo, player, true); // crítico
permitido
        if (player.vida > 0) aplicarAtaquePlayer(player, inimigo);
    }
}

// Turno de cura: inimigo pode atacar de volta, mas SEM chance de
crítico.
private static void turnoCura(Player player, Inimigo inimigo) throws
InterruptedException {
    int inimigoAcertouAtaque = rand.nextInt(0,3);
    player.curar();
    if (inimigoAcertouAtaque == 2){
        digitar("0 " + inimigo.nome + " ataca durante sua cura!");
        aplicarAtaqueInimigo(inimigo, player, false); // crítico
BLOQUEADO
    }
    else {
        digitar("0 " + inimigo.nome + " errou o ataque durante a
cura!");
    }
}

// [COMBATE > Fluxo]

public static boolean ExecutarCombate(Player player, Inimigo inimigo)
throws InterruptedException {
    int round = 1;
    int danoAcumulado = 0;

    digitar("\nApareceu um " + Cores.VERMELHO + inimigo.nome +
Cores.RESET + "!!");

    while (player.vida > 0 && inimigo.vida > 0) {
        int vidaAntes = player.vida;

```

```

        mostrarHud(inimigo, player, round);

        int escolha = lerOpcaoValida(1, 3);

        // Status não consome round
        if (escolha == 3) {
            mostrarStatus(player, danoAcumulado);
            limparTelaDevagar();
            continue;
        }

        if (escolha == 1) turnoAtaque(player, inimigo);
        else                turnoCura(player, inimigo);

        danoAcumulado += Math.max(0, vidaAntes - player.vida);
        Thread.sleep(1500);
        round++;
        limparTela();
    }

    if (player.vida <= 0) {
        digitar("💀 Você morreu...");
        return false;
    }
    digitar(" Você venceu " + Cores.VERMELHO + inimigo.nome +
Cores.RESET + "!");
    Thread.sleep(1500);
    return true;
}

// Encapsula combate + reset de vida + escolha de buff em uma só fase
private static boolean executarFase(Player player, Inimigo inimigo, int
nivelBuff, int salaAtual) throws InterruptedException {
    mostrarSala(salaAtual);
    if (!ExecutarCombate(player, inimigo)) return false;
    player.resetarVida();
    // escolherEAplicarBuff(player, nivelBuff);
    return true;
}

// [GRÁFICOS]

// [GRÁFICOS > HUD]

public static void mostrarHud(Inimigo inimigo, Player player, int round)

```



```

    "┌───────────┐",
    "└───────────┘"
};

int[][] cores = {
    {255, 0, 0},
    {220, 0, 20},
    {190, 0, 40},
    {160, 0, 60},
    {130, 0, 80},
    {100, 0, 100}
};

for (int i = 0; i < titulo_BloodDivine.length; i++) {
    int r = cores[i][0];
    int g = cores[i][1];
    int b = cores[i][2];

    // Define a cor da linha
    System.out.print("\u001B[38;2;" + r + ";" + g + ";" + b + "m");

    // Percorre cada caractere da linha para o efeito de digitação
    String linha = titulo_BloodDivine[i];
    for (int j = 0; j < linha.length(); j++) {
        System.out.print(linha.charAt(j));
        // Velocidade mais rápida (5ms) para não demorar demais,
        // já que o título tem muitos caracteres
        Thread.sleep(5);
    }

    System.out.println(); // Pula para a próxima linha do desenho
}

System.out.println("\u001B[0m"); // Reseta a cor
}

// [GRÁFICOS > Salas]

public static void duasLinhas() {
    limparTela();
    System.out.println("* * * * *");
    System.out.println("* * * * *");
}

public static void apagar() {
```

```
        for (int j = 0; j < 30; j++) {
            System.out.print("\b");
        }
    }

    public static void entrarSala(int sala) throws InterruptedException {
        System.out.println();
        limparTela();
        System.out.println("Entrando na sala " + sala + "...");
        Thread.sleep(1000);
    }

    public static void mostrarSala(int sala) throws InterruptedException {
        switch (sala) {
            case 1 -> {
                duasLinhas();
                entrarSala(1);
                pre_batalha_esqueleto();
                limparTelaDevagar();
            }
            case 2 -> {
                duasLinhas();
                entrarSala(2);
                pre_batalha_goblin();
                limparTelaDevagar();
            }
            case 3 -> {
                duasLinhas();
                entrarSala(3);
                pre_batalha_Demonio();
                limparTelaDevagar();
            }
            case 4 -> {
                duasLinhas();
                entrarSala(4);
                pre_batalha_Cavaleiro_Negro();
                limparTelaDevagar();
            }
            case 5 -> {
                duasLinhas();
                entrarSala(5);
                pre_batalha_rei_demonio();
                limparTelaDevagar();
            }
            default -> { System.out.println("ERRO! Número de sala inválido,
```

```

    apenas de 1 até 5");}
    }
}

// [NARRADOR]

//[NARRADOR > Introdução]
public static void introducao() throws InterruptedException {
    limparTela();
    digitar("Há muitos séculos...");
    Thread.sleep(2500);
    digitar("os Cavaleiros Templários lutavam em nome da Igreja contra
as forças do mal.");
    Thread.sleep(1000);
    digitar("Durante as guerras, a Igreja do Calvário foi destruída e o
hoje permanece abandonada...");
    Thread.sleep(2000);
    digitar("Para purificar o templo, um nobre Cavaleiro adentra as
escadarias da igreja...");
    Thread.sleep(2000);
}

//[NARRADOR > Pre-Batalha]

public static void pre_batalha_esqueleto() throws InterruptedException {
    limparTela();
    digitar("O Cavaleiro, imerso na escuridão...");
    digitar("Acende uma tocha revelando ossos espalhados pela sala");
    Thread.sleep(1000);
    digitar("Ao som das gotas das paredes úmidas...
");
    digitar("Os ossos da caverna começam a se juntar...
");
    digitar("O inimigo é um " + Inimigos.ESQUELETO + "!");
    Thread.sleep(2200);
}

public static void pre_batalha_goblin() throws InterruptedException {
    digitar("De uma silhueta escura se avista uma pele esverdeada...");
    Thread.sleep(1000);
    digitar("A figura ronda o Cavaleiro pela escuridão esperando a
melhor oportunidade de ATACAR!");
    digitar("O inimigo é um " + Inimigos.GOBLIN + "!");
    Thread.sleep(2200);
}

```

```
public static void pre_batalha_Demonio() throws InterruptedException {
    limparTela();
    digitar("As paredes de pedra mascaradas com sangue revelam um
convecionário");
    Thread.sleep(2000);
    digitar("No teto, um vulto, algo está se mexendo e muito
rápido...");
    Thread.sleep(1000);
    digitar("Uma criatura de asas negras desce em sua direção
");
    Thread.sleep(1000);
    digitar("O inimigo é um " + Inimigos.DEMONIO + "!");
    Thread.sleep(2200);
}
```

```
public static void pre_batalha_Cavaleiro_Negro() throws
InterruptedException {
    limparTela();
    digitar("Ao centro de um grande salão iluminado por velas...");
    Thread.sleep(1500);
    digitar("Uma figura imponente se ergue, vestida com uma armadura
negra reluzente...");
    Thread.sleep(1500);
    digitar("E com um único golpe");
    digitar("Destroi a imagem enfurecendo o Cavaleiro");
    Thread.sleep(2000);
    digitar("O inimigo é o " + Inimigos.CAVALEIRO_NEGRO + "!");
    Thread.sleep(2200);
}
```

```
public static void pre_batalha_rei_demonio() throws InterruptedException
{
    limparTela();
    digitar("Em instantes");
    Thread.sleep(1000);
    digitar("A escuridão que cobria a igreja...");
    Thread.sleep(1500);
    digitar("Foi tomada por um fogo que envolvia um Grande Trono Do
Castelo Sombrio");
    Thread.sleep(2000);
    digitar("De lá, o Rei Demônio aguarda sua chegada...");
    Thread.sleep(2200);
    digitar("O inimigo é o " + Inimigos.REI_DEMONIO + "!");
    Thread.sleep(600);
    digitar("Se prepare Cavaleiro!");
    digitar("A batalha final se aproxima... Este é o desafio supremo!");
}
```

```

✂ ");
    Thread.sleep(2200);
}

//[NARRADOR > Pos-Batalha]

public static void pos_batalha_Esqueleto() throws InterruptedException {
    limparTela();
    digitar("O ESQUELETO foi derrotado");
    Thread.sleep(1500);
    digitar("Mas o que o Cavaleiro havia enfrentado de fato?");
    digitar("Seria essa criatura fruto de sua imaginação...");
    Thread.sleep(1500);
    digitar("Ele temia que não!");
    Thread.sleep(2500);
}

public static void pos_batalha_Goblin() throws InterruptedException {
    limparTela();
    digitar("O cravejar da lâmina do Cavaleiro ecoa como um último som
antes do silêncio...");
    Thread.sleep(2000);
    digitar("aquele silêncio deixava claro...");
    Thread.sleep(1000);
    digitar("A Igreja do Calvário era um templo das trevas");
    Thread.sleep(2200);
}

public static void pos_batalha_Demonio() throws InterruptedException {
    limparTela();
    digitar("O Cavaleiro sentia um aperto em seu coração");
    Thread.sleep(1500);
    digitar("Como poderam manchar o nome da Igreja? Ele se perguntava");
    Thread.sleep(1500);
    digitar("O Cavaleiro havia entendido o seu propósito...");
    Thread.sleep(1500);
    digitar("Limpar o legado da Igreja do Calvário e derrotar todas as
criaturas profanas no templo!");
    Thread.sleep(3000);
}

public static void pos_batalha_Cavaleiro_Negro() throws
InterruptedException {
    limparTela();
    digitar("O Cavaleiro atravessou a câmara em silêncio");
}

```

```
        Thread.sleep(1500);
        digitar("e continuou avançando pelas profundezas da antiga
igreja...");
        Thread.sleep(1500);
        digitar("Mal sabia ele que um grande desafio o aguardava na próxima
sala");
        Thread.sleep(2500);
    }

    public static void finalDoJogo() throws InterruptedException{
        limparTela();
        digitar("O Rei Demônio foi derrotado!");
        digitar("O mundo está a salvo!");
        Thread.sleep(1500);
        digitar("O Cavaleiro havia cumprido seu destino.");
        Thread.sleep(1500);
        digitar("Agora ele pode descansar em paz...");
        Thread.sleep(1500);
        digitar("★BLOODIVINE COMPLETO! Parabéns, herói! ★");
    }
}
```



## 5 Conclusão

Este projeto consiste na construção de um sistema completo de RPG em Java, desenvolvido com o objetivo de simular uma experiência autêntica de batalha por turnos diretamente no terminal. A proposta central é oferecer uma jogabilidade funcional, imersiva e direta, na qual o jogador assume o controle de um herói, escolhe uma classe entre três opções distintas e enfrenta um inimigo em um duelo intenso de vida ou morte.

Apesar de ser um projeto compacto, ele demonstra de forma prática a aplicação de conceitos importantes de programação orientada a objetos, lógica estruturada, controle de fluxo e boas práticas de organização de código. Logo na inicialização, o sistema exibe um menu interativo de seleção de classe, marcando o primeiro nível de imersão do jogador. As opções disponíveis — Guerreiro, Assassino e Tank — não são meros nomes decorativos. Cada classe possui atributos únicos de vida máxima, dano base, quantidade de poções e estilo de gameplay. Essa personalização é implementada por meio da classe `Player`, que contém um método construtor inteligente (`criarPersonagem()`) responsável por receber a escolha do usuário via `Scanner` e retornar um objeto já configurado com os valores correspondentes. Essa abordagem centraliza a lógica de criação do personagem, facilita a manutenção e evita duplicação de código, sendo especialmente didática para quem está aprendendo Java. A interação com o usuário foi projetada com foco em robustez e experiência. Utilizando a classe `Scanner`, o programa mantém um diálogo constante com o jogador. Para evitar crashes por entradas inválidas, foi implementada uma validação eficiente com `hasNextInt()` combinada com loops de repetição, garantindo que o sistema só avance quando receber dados corretos. Além disso, o uso de códigos ANSI para coloração do terminal eleva consideravelmente a experiência visual: danos causados aparecem em vermelho vibrante, curas em verde, informações importantes em amarelo e mensagens de vitória/derrota com destaques especiais. Esses pequenos detalhes transformam um simples jogo de console em algo muito mais atraente e profissional. O núcleo do sistema é o loop de combate, controlado por uma estrutura `while` que continua ativa enquanto tanto o jogador quanto o inimigo possuírem vida acima de zero. Dentro desse ciclo principal, o jogador pode escolher entre três ações principais: Atacar, Curar ou Ver Status. O ataque não se resume a uma simples subtração — ele envolve cálculo de dano (com possibilidade de expansão futura para dano crítico ou aleatório), atualização da vida do oponente, exibição de mensagens estilizadas e verificação imediata de morte. Caso o inimigo sobreviva, ele realiza um contra-ataque automático, criando um verdadeiro vai-e-vem estratégico. A mecânica de cura adiciona profundidade tática ao jogo. O jogador começa com um número limitado de poções de vida, forçando-o a tomar decisões importantes: deve arriscar tudo no ataque para finalizar o combate rapidamente ou

utilizar recursos preciosos para sobreviver mais turnos? Essa limitação de recursos cria tensão e replayability, características essenciais em bons RPGs. O inimigo é tratado como uma entidade separada, representada por uma classe `Enemy` com atributos como nome, vida, dano e uma breve descrição. A função geradora de inimigos foi construída de forma modular, permitindo fácil expansão. Com poucas modificações, é possível implementar uma lista de monstros variados (esqueletos, orcs, dragões, magos sombrios etc.), cada um

com comportamentos, fraquezas ou ataques especiais diferentes. Isso demonstra maturidade no design do código, priorizando escalabilidade e reutilização. Além das mecânicas centrais, o projeto investe significativamente na experiência do usuário (UX).

Dois métodos utilitários se destacam: o `digitar()`, que simula o efeito de máquina de escrever processando o texto caractere por caractere com pausas controladas por `Thread.sleep()`, trazendo uma nostalgia dos clássicos RPGs dos anos 80 e 90; e o `limparTela()`, que mantém o console organizado, evitando que o histórico de turnos anteriores polua a tela e distraindo o jogador. Outro aspecto relevante é a organização geral do código. O projeto utiliza pacotes bem definidos, classes separadas para cada responsabilidade (`Player`, `Enemy`, `Game`, `Utils`), métodos com nomes claros e comentários estratégicos. Essa estrutura torna o código legível, manutenível e fácil de ser expandido por outros desenvolvedores ou pelo próprio autor no futuro. Em termos de aprendizado, este projeto é extremamente rico. Ele permite praticar conceitos como: herança e polimorfismo (caso queira evoluir as classes), encapsulamento, tratamento de exceções, manipulação de strings, controle de fluxo avançado, e até mesmo noções básicas de design patterns (como `Factory` para criação de inimigos). Além disso, trabalhar com terminal reforça o entendimento de como o Java se comunica com o ambiente de execução.

Futuramente, o sistema possui grande potencial de evolução: implementação de múltiplos inimigos em sequência, sistema de níveis e experiência, diferentes mapas/áreas, salvamento de progresso com arquivos, interface gráfica com JavaFX ou Swing, sons, balanceamento mais sofisticado, e até modo multiplayer local (hotseat). Em resumo, este RPG de terminal não é apenas um exercício acadêmico. É um jogo funcional, divertido e bem construído que demonstra domínio de fundamentos de programação em Java, criatividade na criação de mecânicas e atenção aos detalhes que fazem a diferença na experiência do usuário. Um excelente projeto para portfólio, tanto para estudantes quanto para desenvolvedores juniores que desejam mostrar habilidades práticas de desenvolvimento de jogos e organização de código.